

Sequence-to-Sequence (Seq 2 Seq) Modeling & Attention

- 1) parallelism not possible
- 2) vanish gradient
- 3) Boldenuek problem

- Many to Many
- one to Many
- caption generation
- Many to One

English → Hindi



Summarization

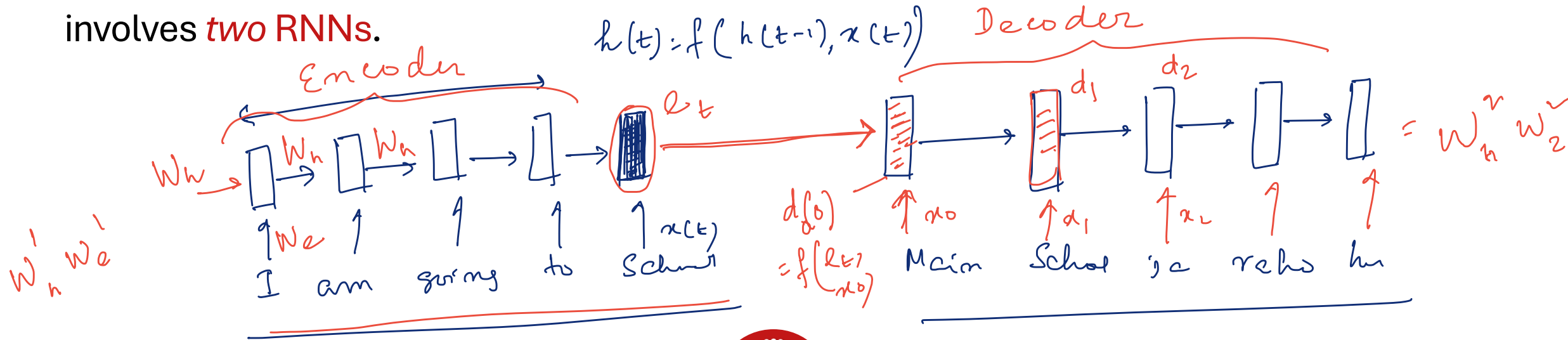




Tanmoy Chakraborty
Associate Professor, IIT Delhi
<https://tanmoychak.com/>

Neural Machine Translation?

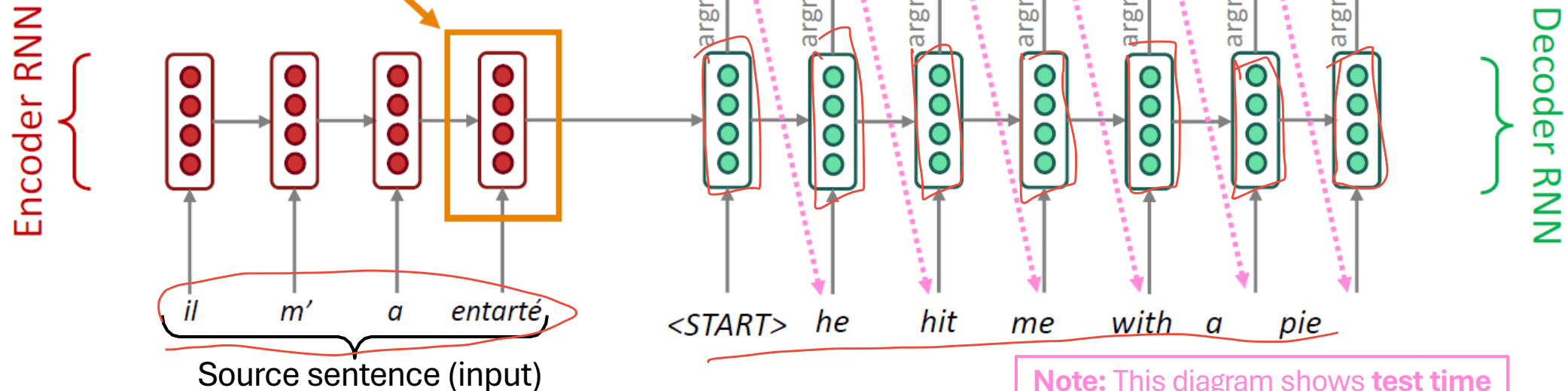
- **Neural Machine Translation (NMT)** is a way to do Machine Translation with a *single neural network*.
- The neural network architecture is called **sequence-to-sequence** (aka **seq2seq**) and it involves **two RNNs**.



Neural Machine Translation (NMT)

The Sequence-to-Sequence Model

Encoding of the source sentence.
Provides initial hidden state
for Decoder RNN.



Encoder RNN produces an *encoding* of the source sentence.

Decoder RNN is a Language Model that generates target sentence, *conditioned on encoding*.

Note: This diagram shows **test time** behavior: decoder output is fed in as next step's input



Neural Machine Translation (NMT)

- The **sequence-to-sequence model** is an example of a **Conditional Language Model**
 - **Language Model** because the decoder is predicting the next word of the target sentence y
 - **Conditional** because its predictions are also conditioned on the source sentence x
- NMT directly calculates $P(y|x)$

$$P(y|x) = P(y_1|x) P(y_2|y_1, x) P(y_3|y_1, y_2, x) \dots \underbrace{P(y_T|y_1, \dots, y_{T-1}, x)}$$

Probability of next target word, given
target words so far and source sentence x

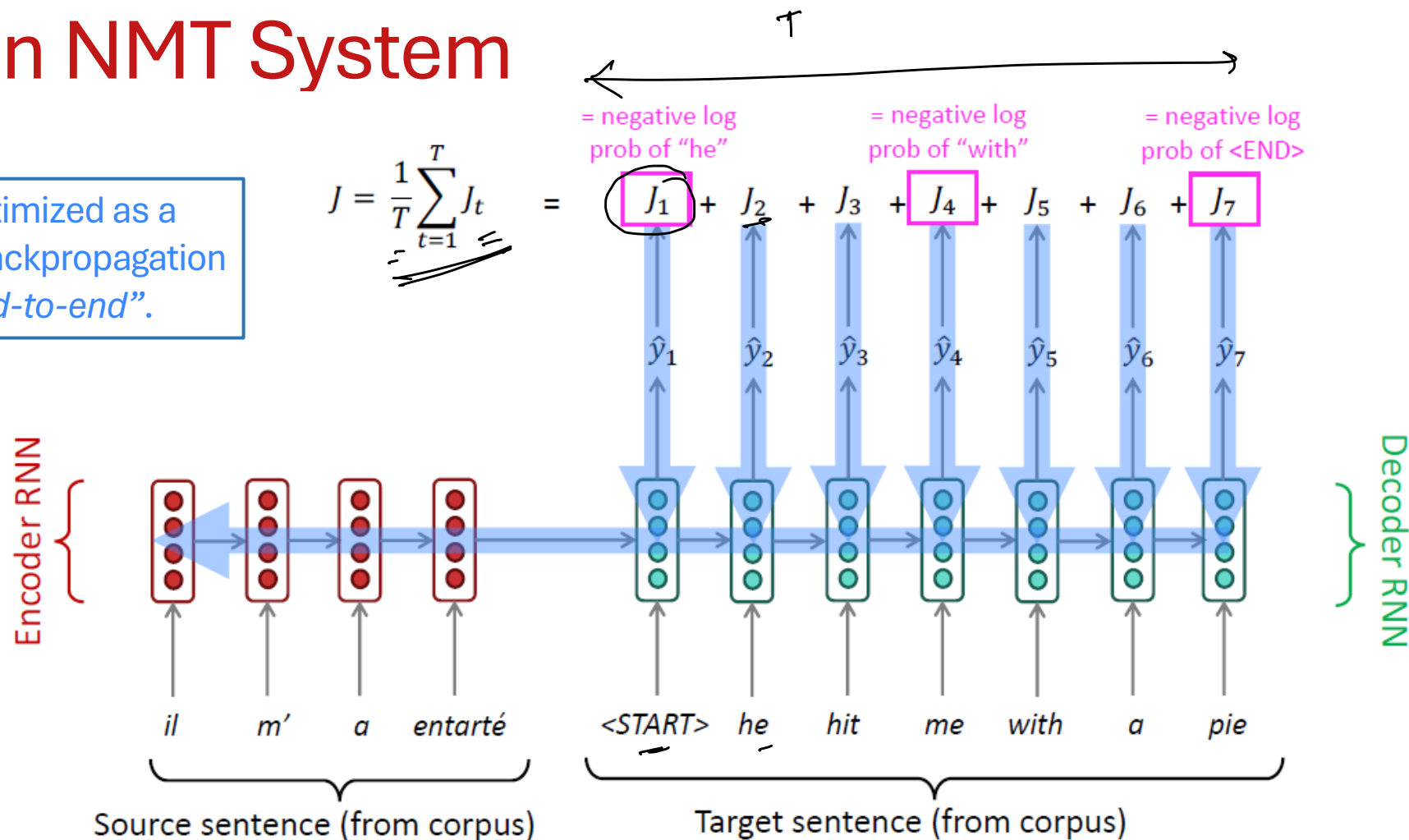
- How to train an NMT system?



Training an NMT System

Seq2seq is optimized as a **single system**. Backpropagation operates “*end-to-end*”.

$$J = \frac{1}{T} \sum_{t=1}^T J_t$$



Teacher forcing

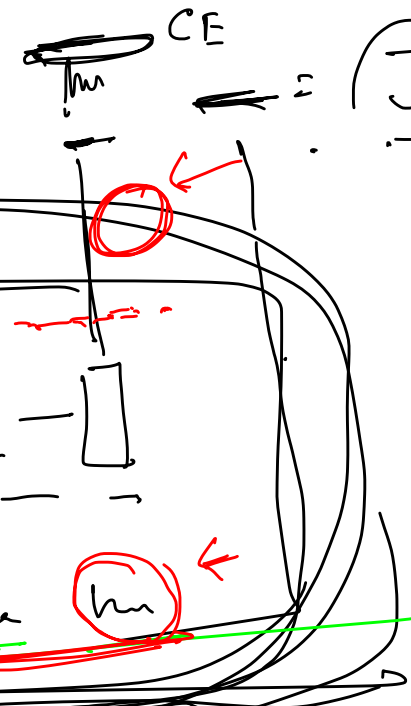
Normal
Dendy

$$J = \frac{1}{N} \sum_i J_i$$

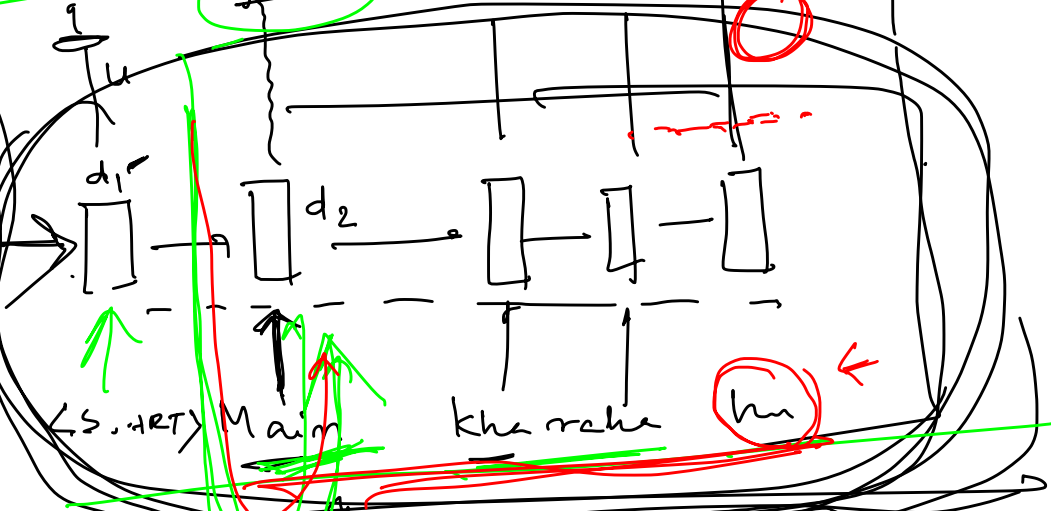
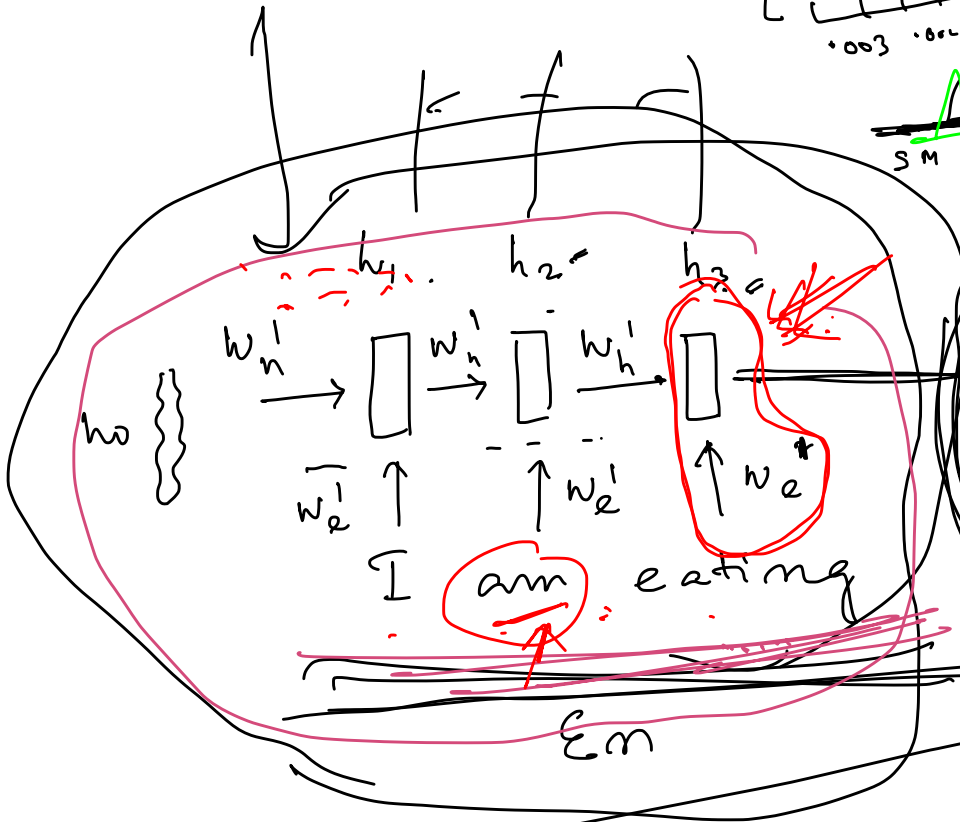
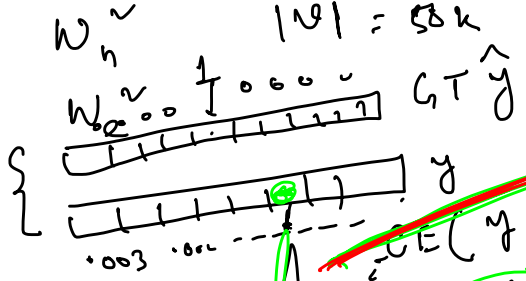
$$\frac{\partial J}{\partial w_h^2} \quad \frac{\partial J}{\partial w_e^2}$$

$$\frac{\partial J}{\partial w_h^1}$$

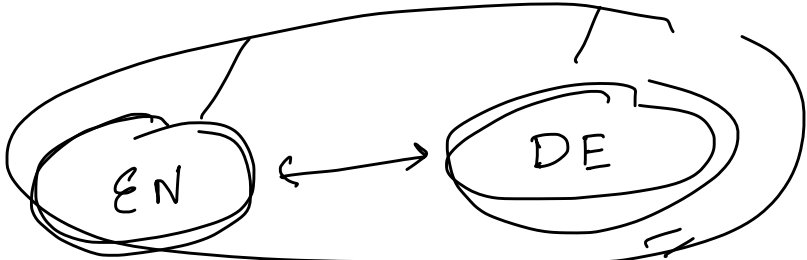
$$\frac{\partial J}{\partial w_e^1}$$

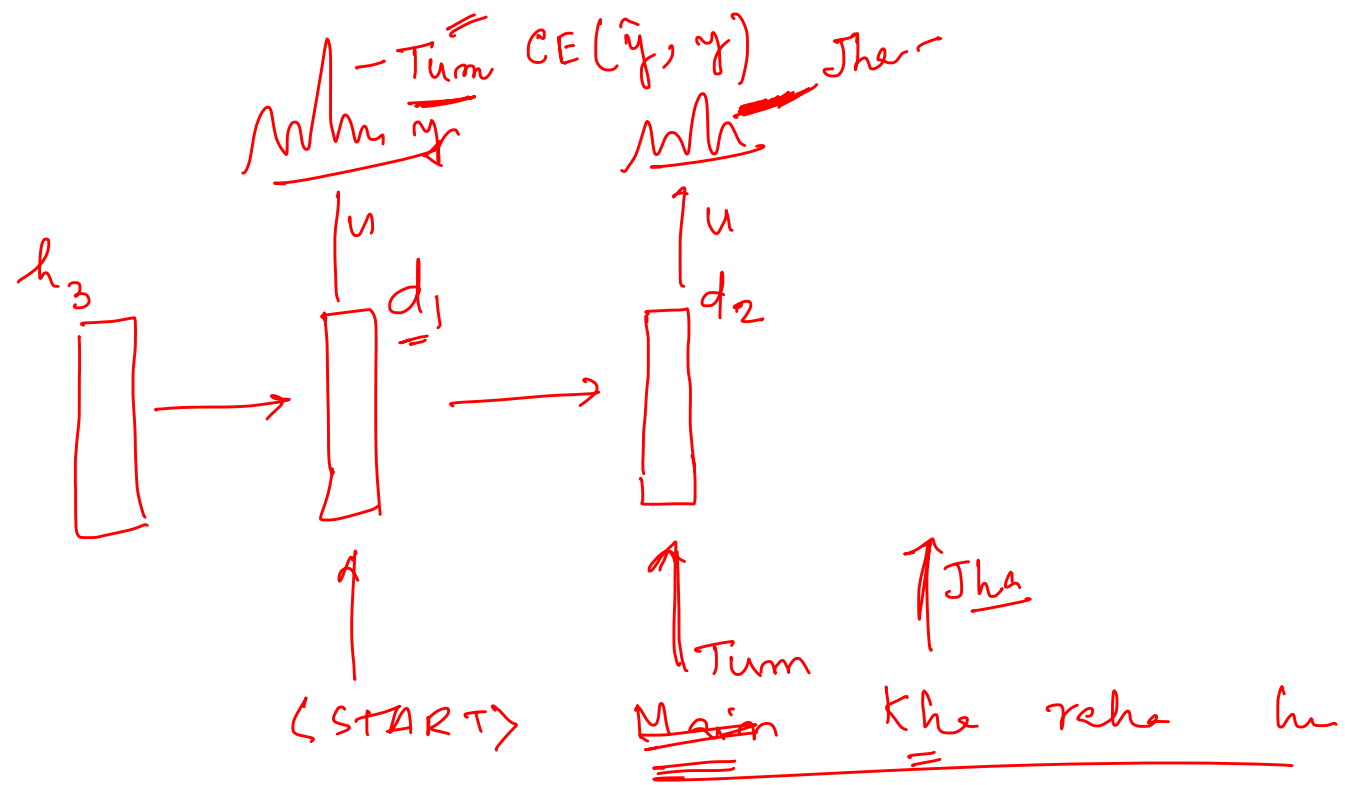


Turn



$$\begin{aligned} \tilde{w}_h^1 &\leftarrow w_h^1 + \eta \frac{\partial J}{\partial w_h^1} \\ \tilde{w}_e^1 &\leftarrow w_e^1 + \eta \frac{\partial J}{\partial w_e^1} \\ \tilde{w}_h^2 &\leftarrow w_h^2 + \eta \frac{\partial J}{\partial w_h^2} \\ \tilde{w}_e^2 &\leftarrow w_e^2 + \eta \frac{\partial J}{\partial w_e^2} \end{aligned}$$

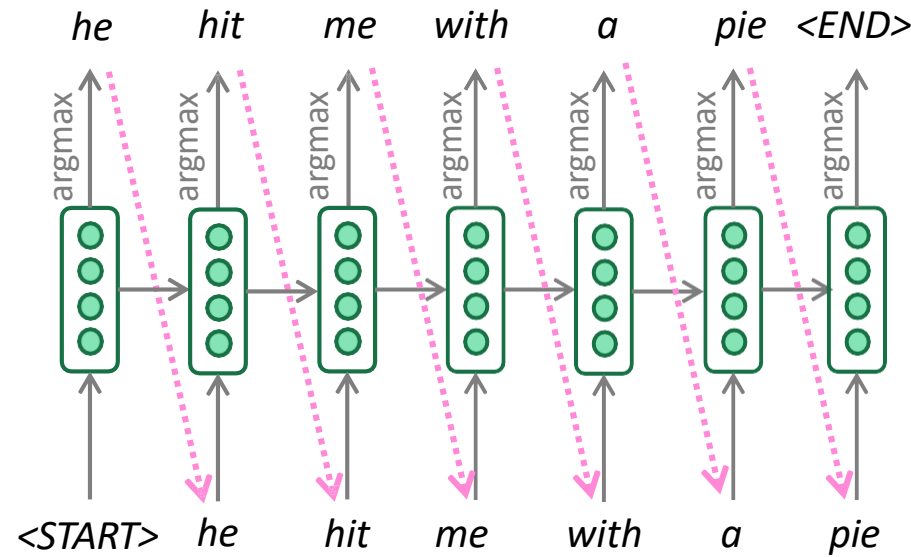




TF
ND
=

Greedy decoding

- We saw how to generate (or “decode”) the target sentence by taking argmax on each step of the decoder.



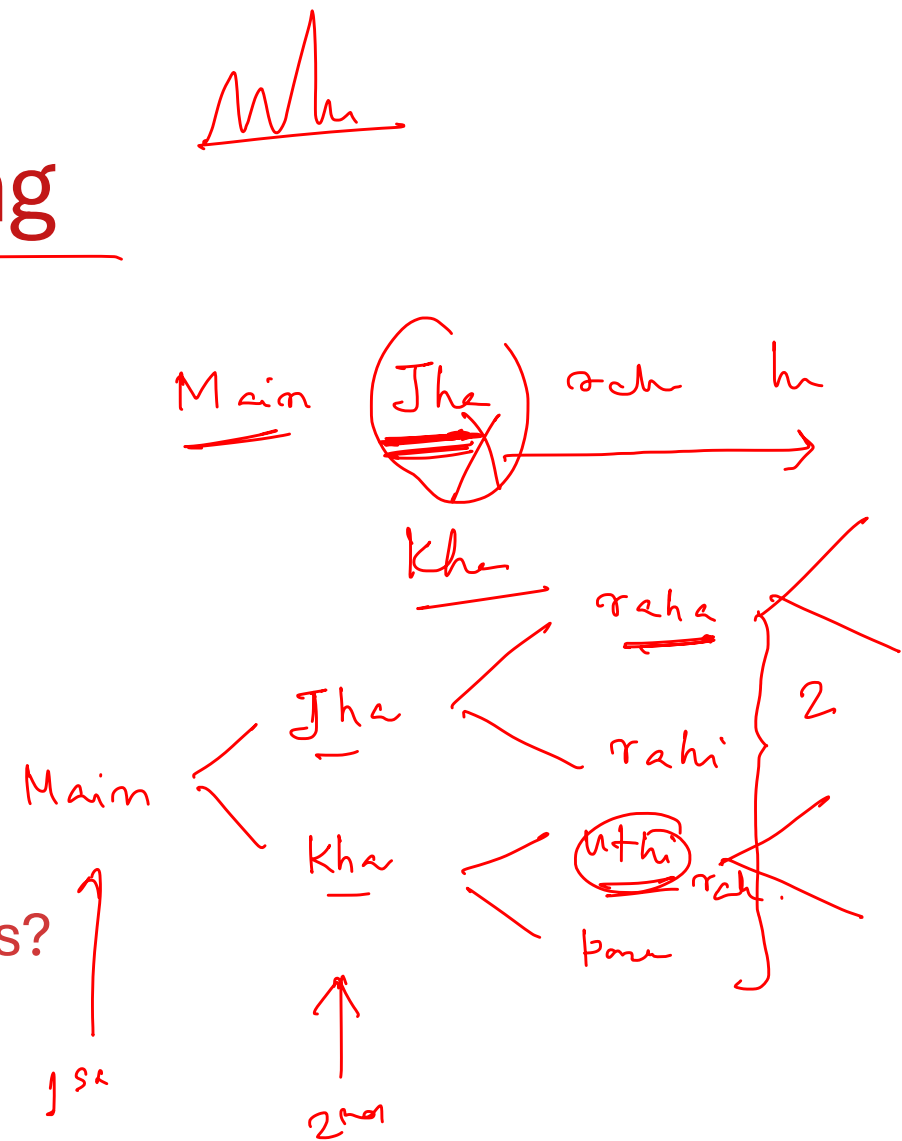
- This is **greedy decoding** (take most probable word on each step)
- **Problems with this method?**



Problems With Greedy Decoding

- Greedy decoding has no way to undo decisions!
- Input: *il a m'entarté* (he hit me with a pie)
- → he _____
- → he hit _____
- → he hit **a** _____ (whoops! no going back now...)

How to fix this?



Exhaustive Search Decoding

- Ideally we want to find a (length T) translation y that maximizes

$$\begin{aligned} P(y|x) &= P(y_1|x) P(y_2|y_1, x) P(y_3|y_1, y_2, x) \dots, P(y_T|y_1, \dots, y_{T-1}, x) \\ &= \prod_{t=1}^T P(y_t|y_1, \dots, y_{t-1}, x) \end{aligned}$$

- We could try computing **all possible sequences** y
- This means that on each step t of the decoder, we're tracking V^t possible partial translations, where V is vocab size
- This $O(V^T)$ complexity is **far too expensive!**



Beam Search Decoding

- **Core idea:** On each step of decoder, keep track of the ***k most probable*** partial translations (which we call ***hypotheses***)
 - k is the **beam size** (in practice around 5 to 10)
- A hypothesis $y_1, \dots, y_t$ has a **score** which is its **log probability**:

$$\text{score}(y_1, \dots, y_t) = \log P_{\text{LM}}(y_1, \dots, y_t | x) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$$

- Scores are all negative, and higher score is better
 - We search for high-scoring hypotheses, tracking top k on each step
- Beam search is **not guaranteed** to find optimal solution
 - But **much more efficient** than exhaustive search!



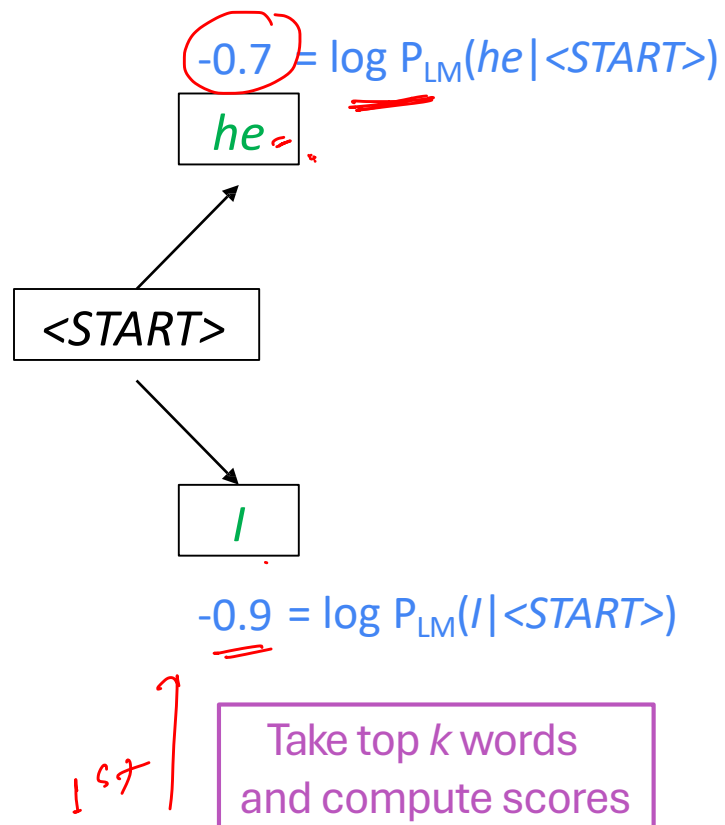
Beam Search Decoding: Example

Beam size = $k = 2$.

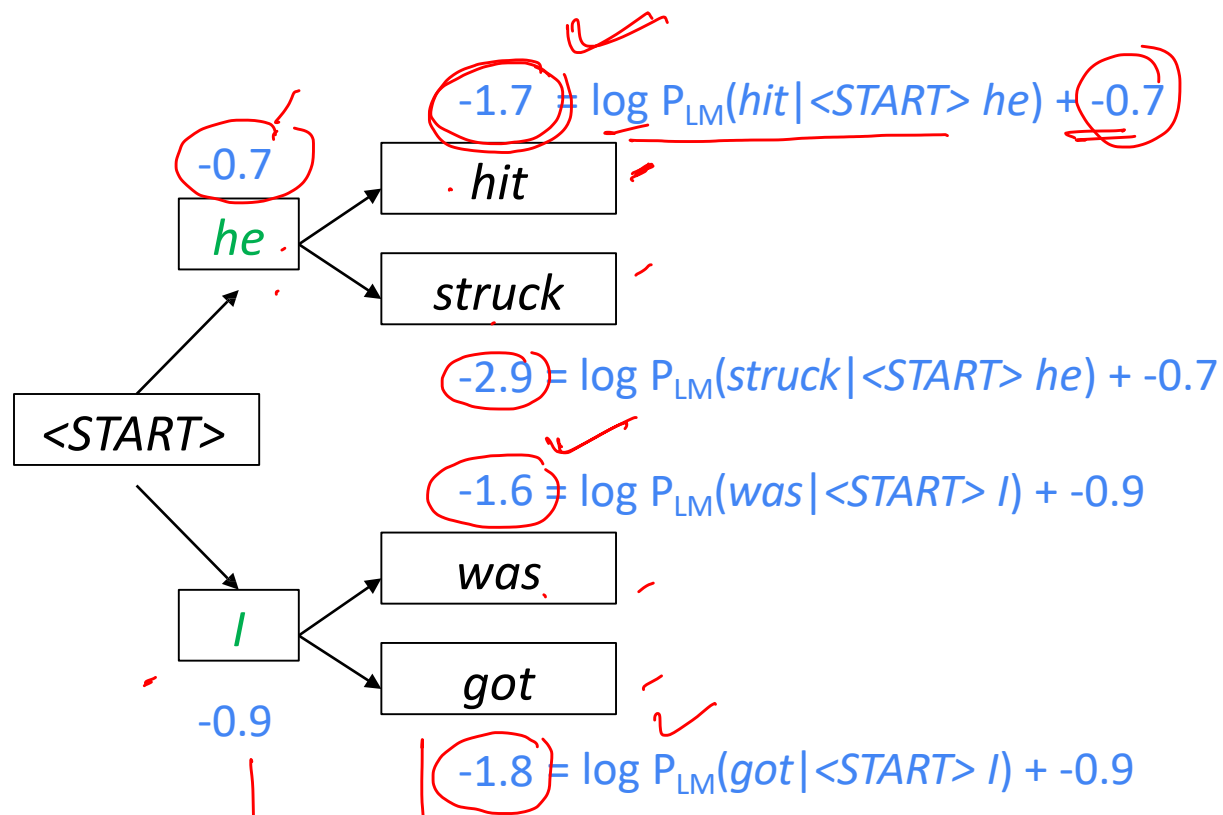
<START>

Calculate prob
distribution of next word

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$

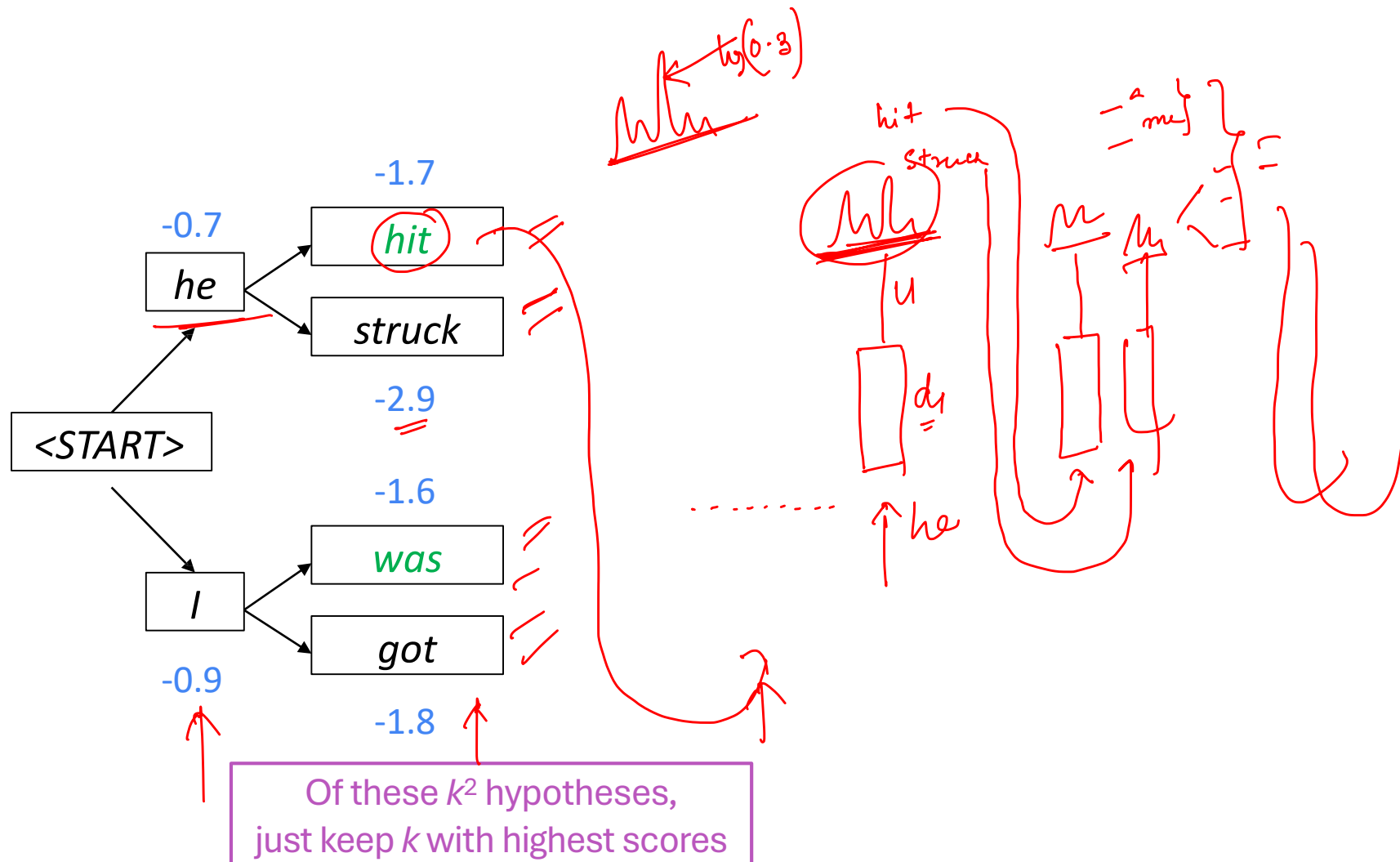


Beam size = k = 2. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$

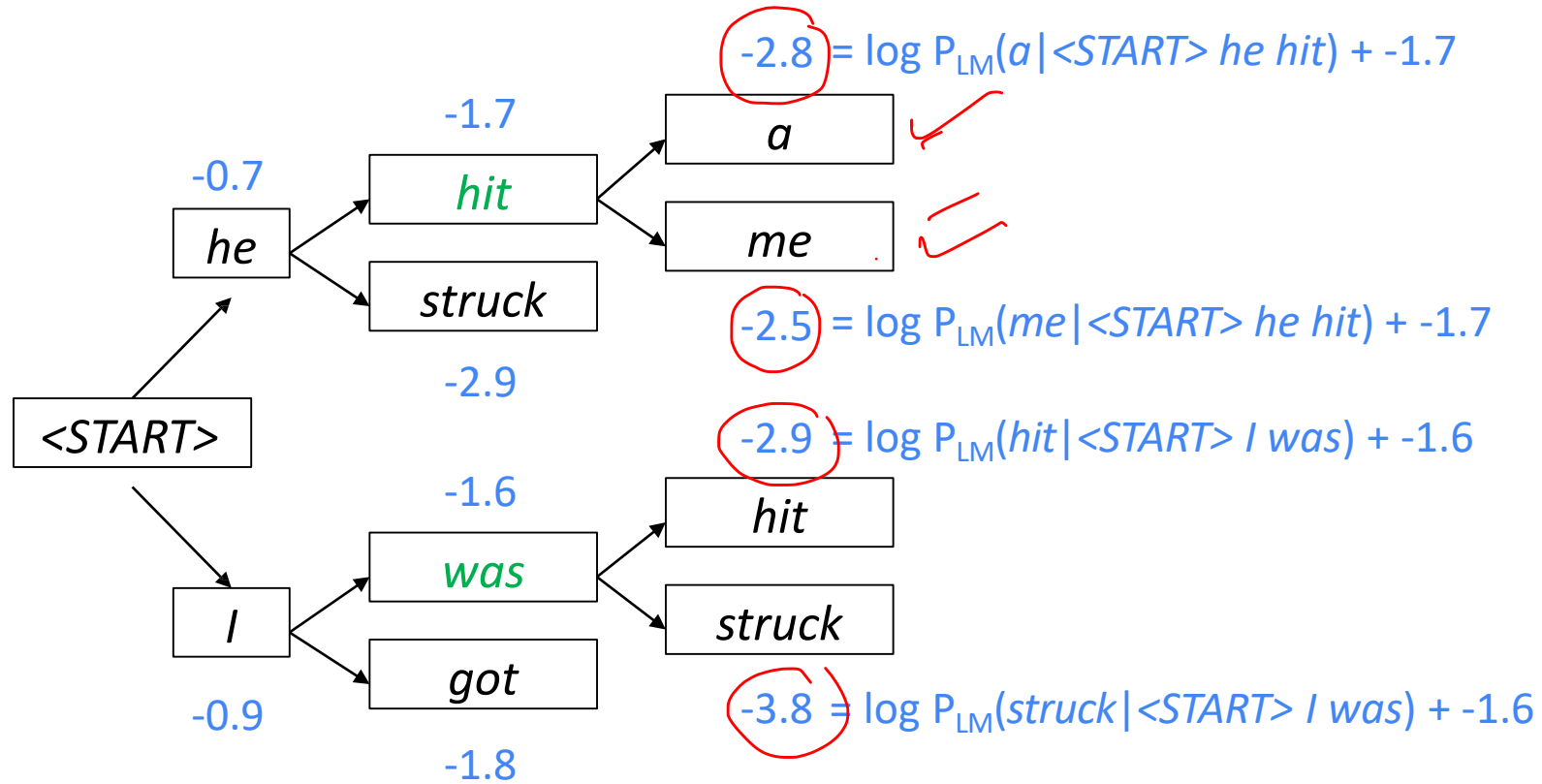


For each of the k hypotheses, find top k next words and calculate scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$

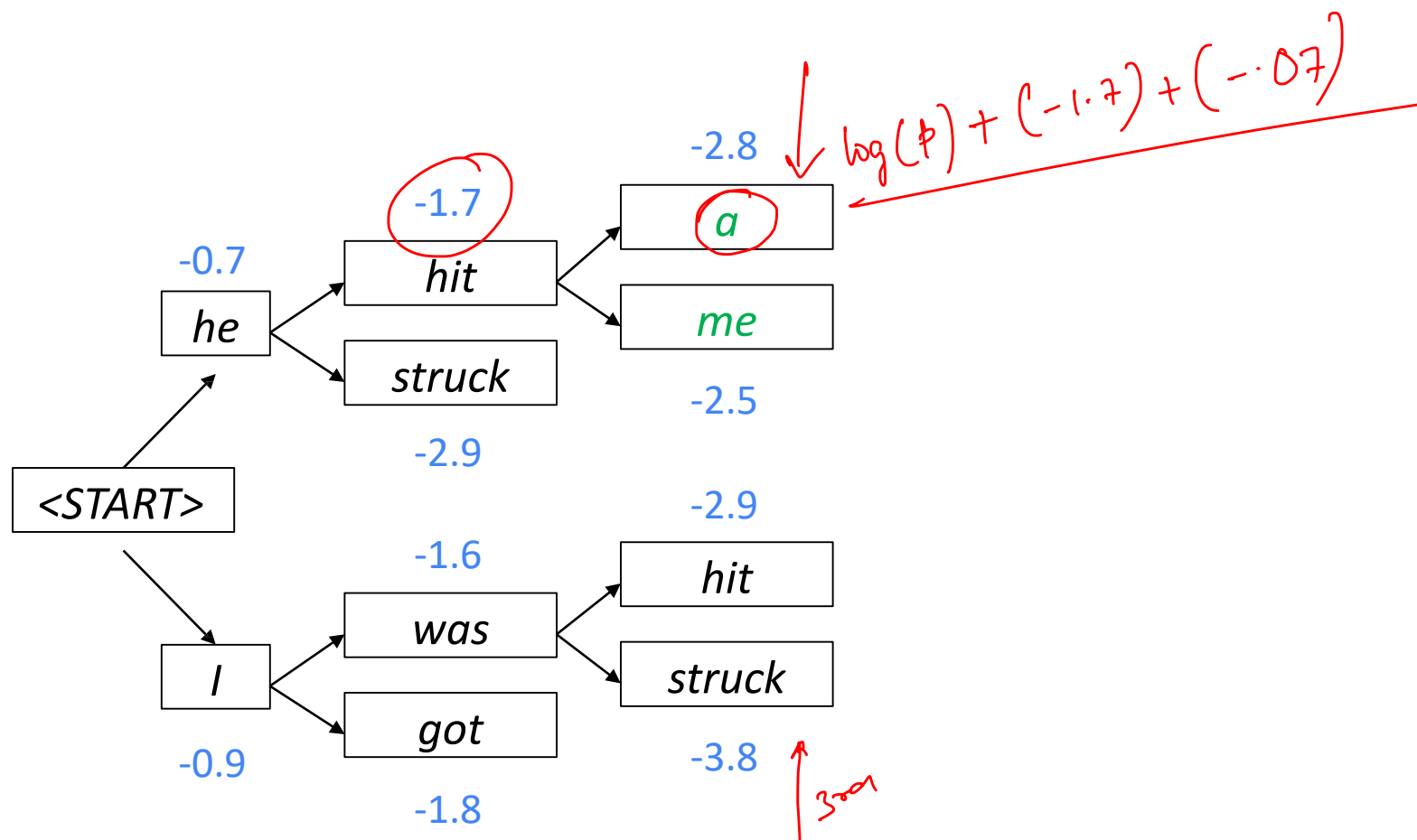


Beam size = k = 2. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



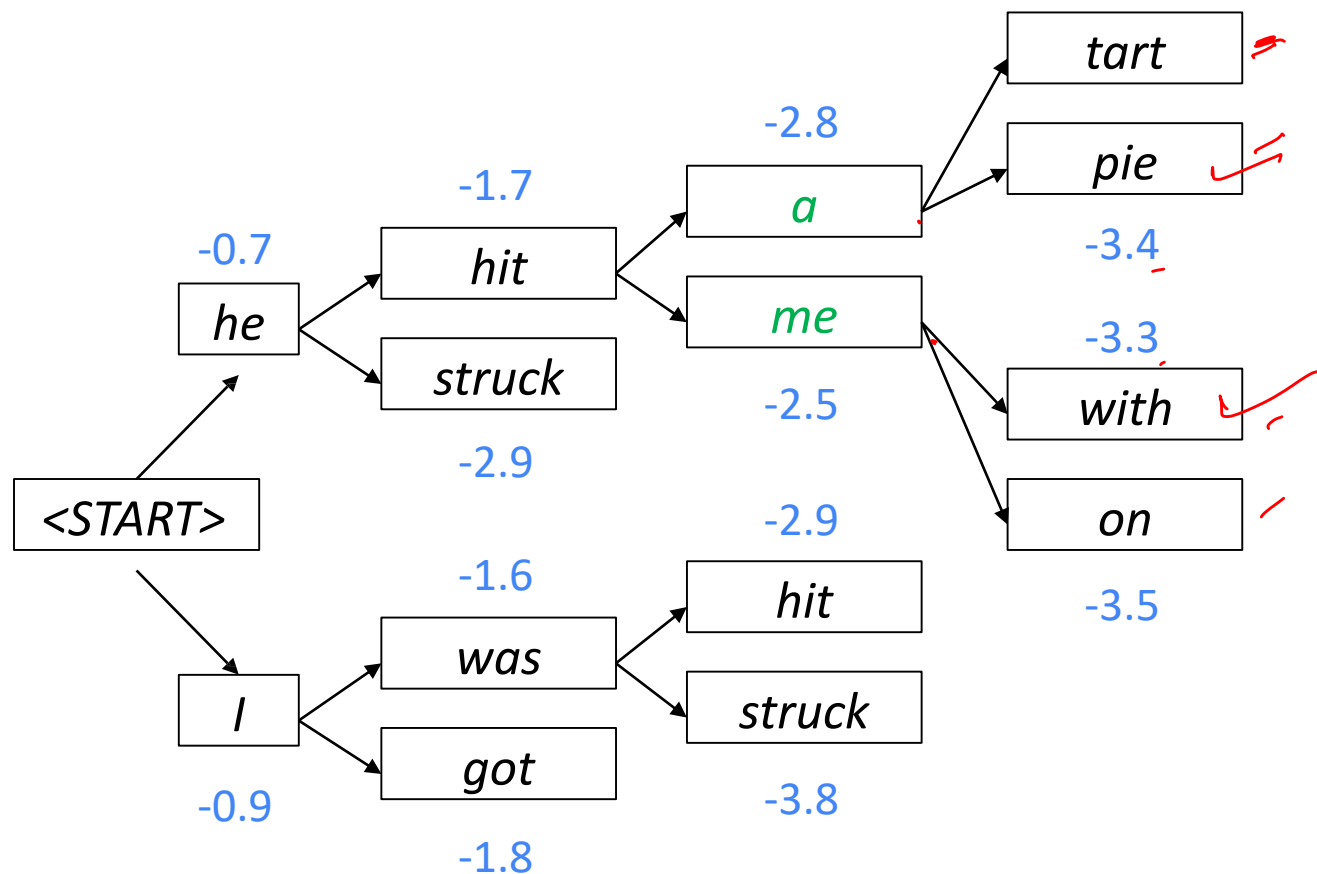
For each of the k hypotheses, find top k next words and calculate scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



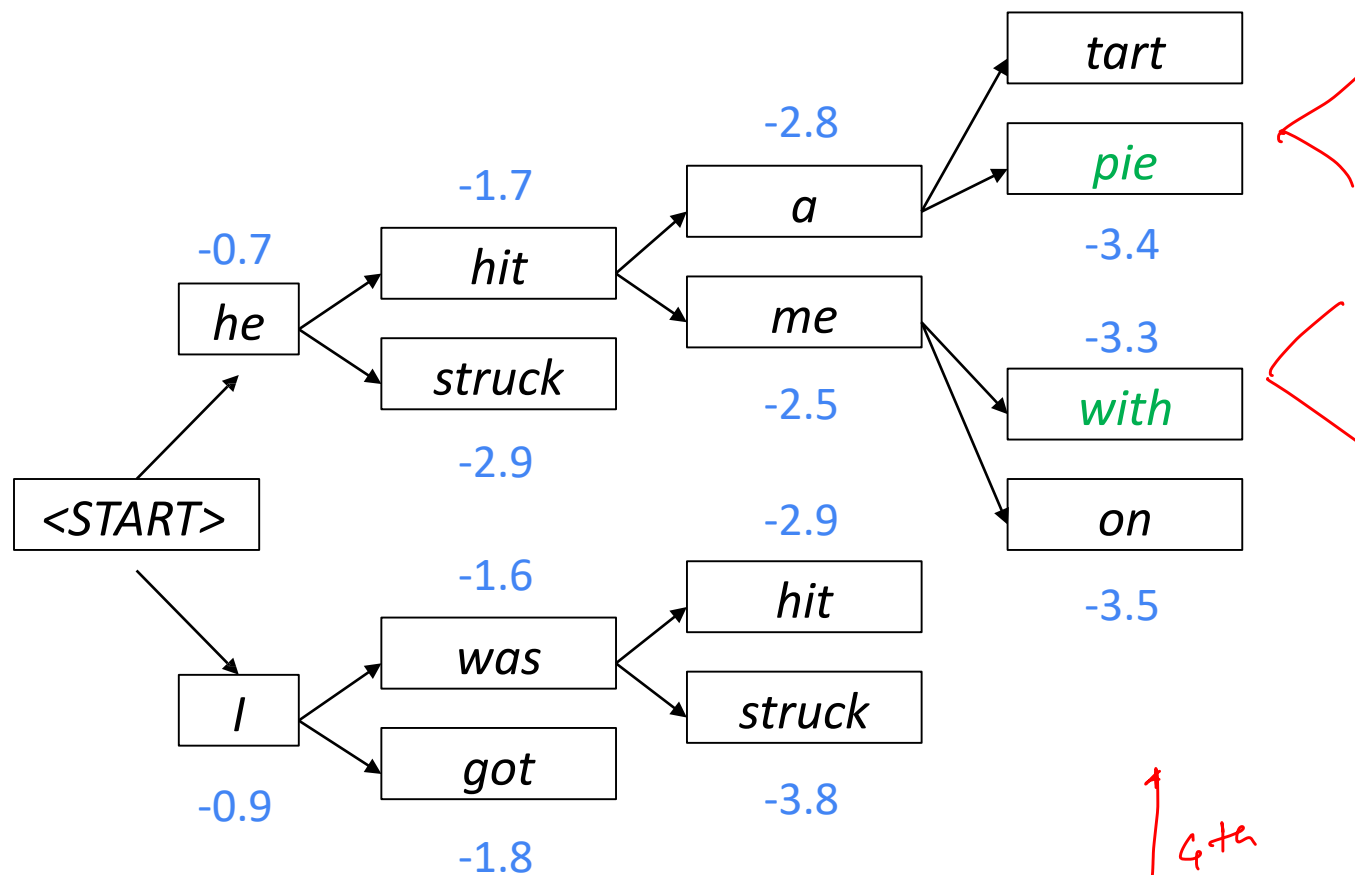
Of these k^2 hypotheses,
just keep k with highest scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



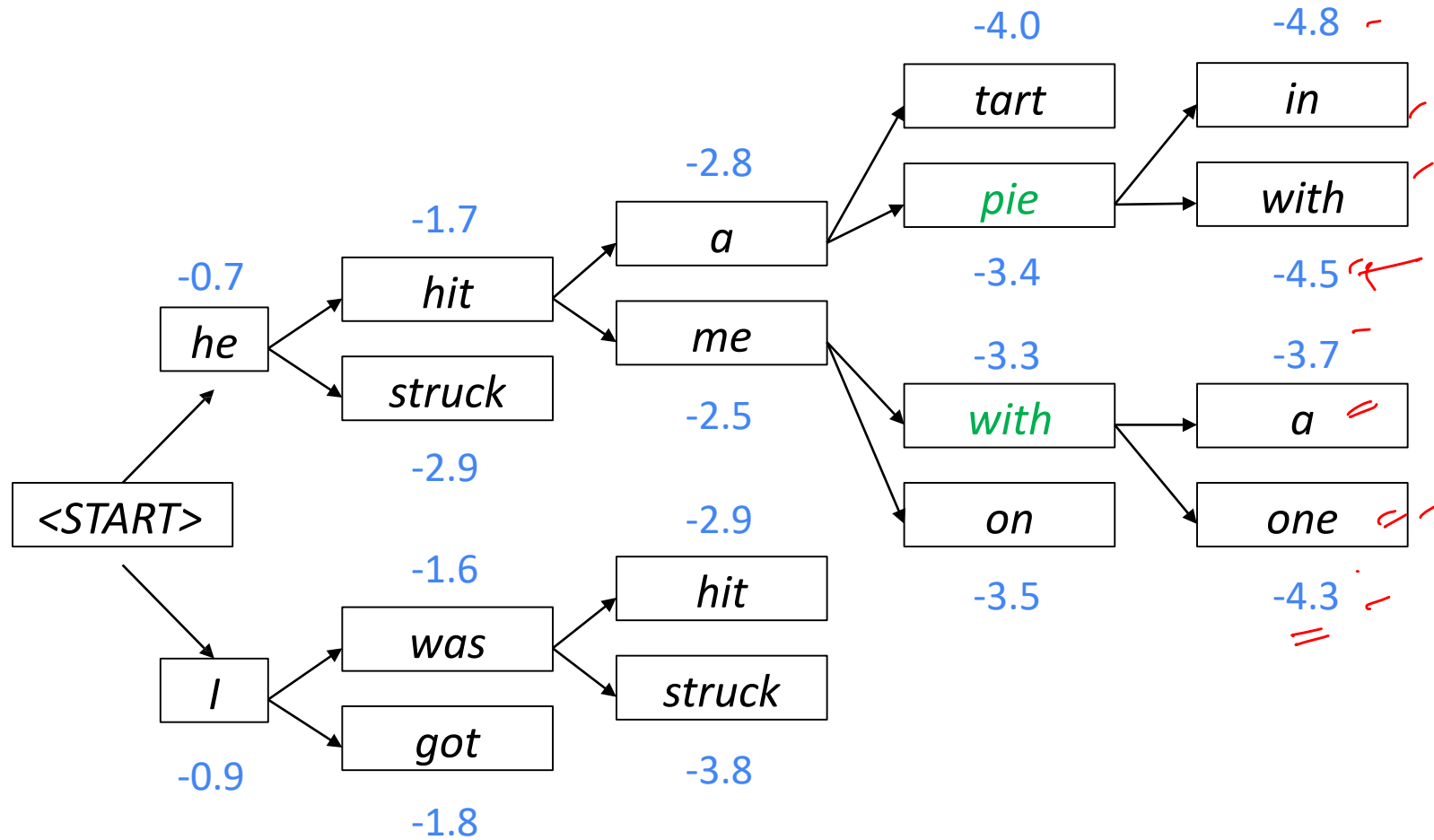
For each of the k hypotheses, find top k next words and calculate scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



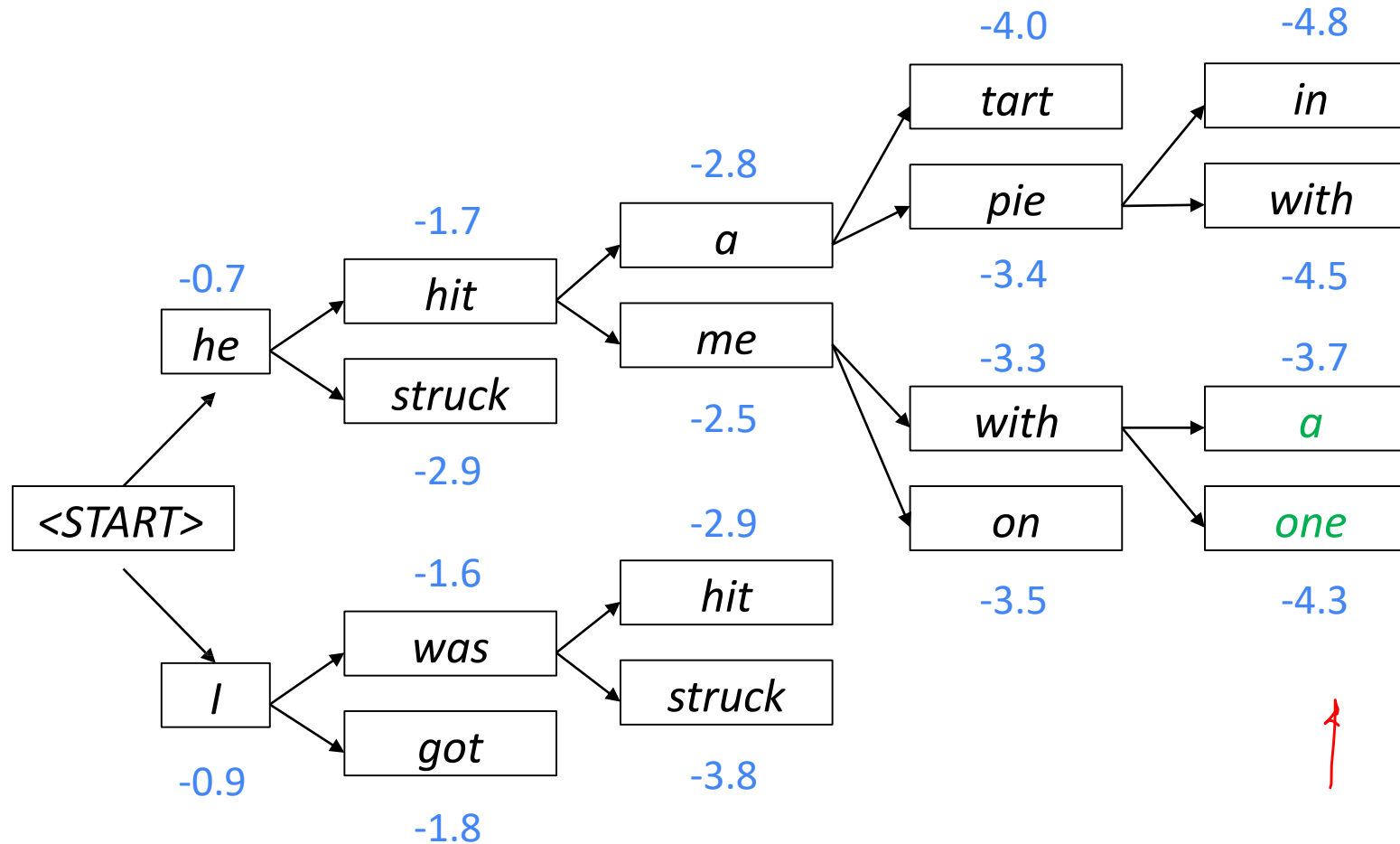
Of these k^2 hypotheses,
just keep k with highest scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



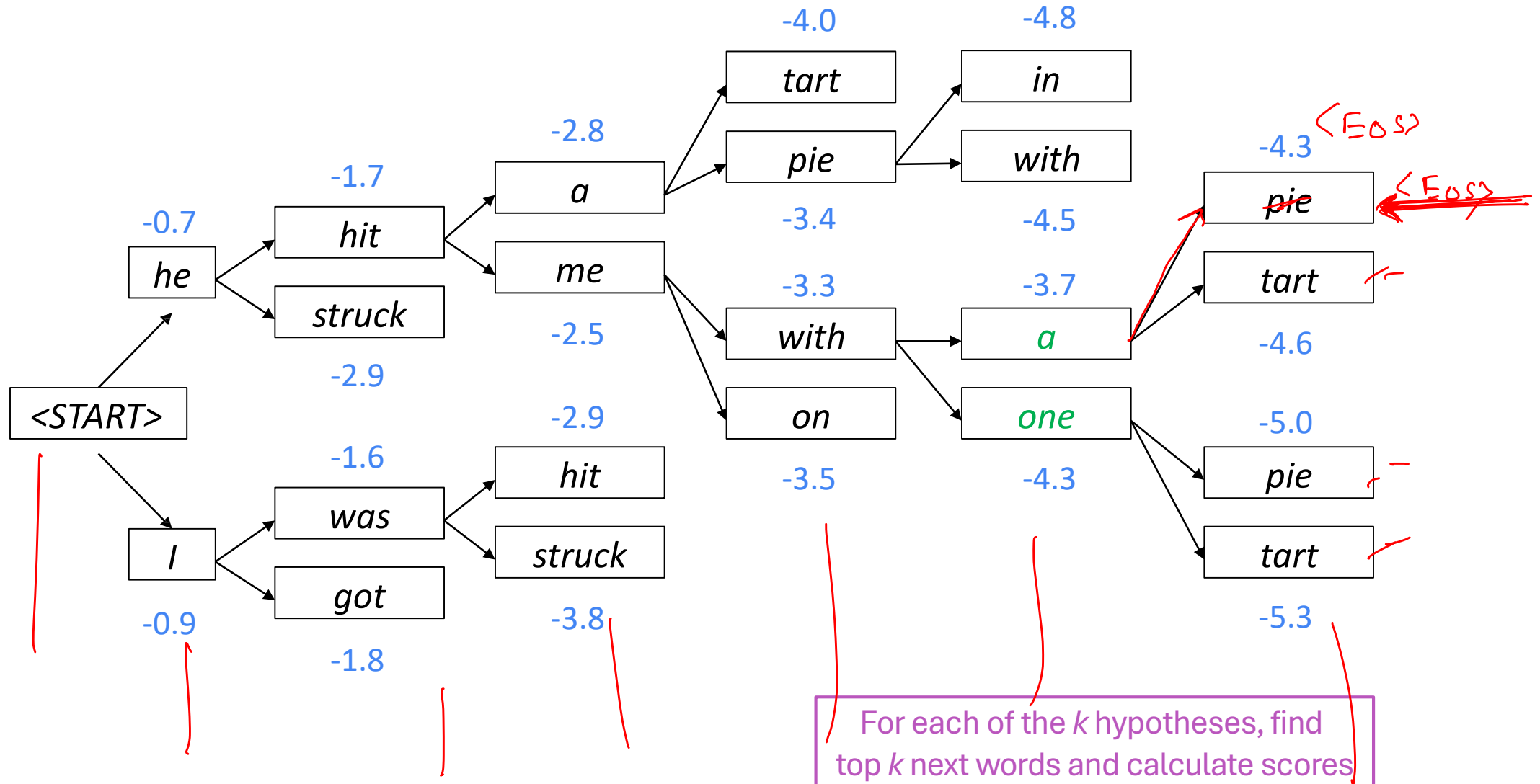
For each of the k hypotheses, find top k next words and calculate scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$

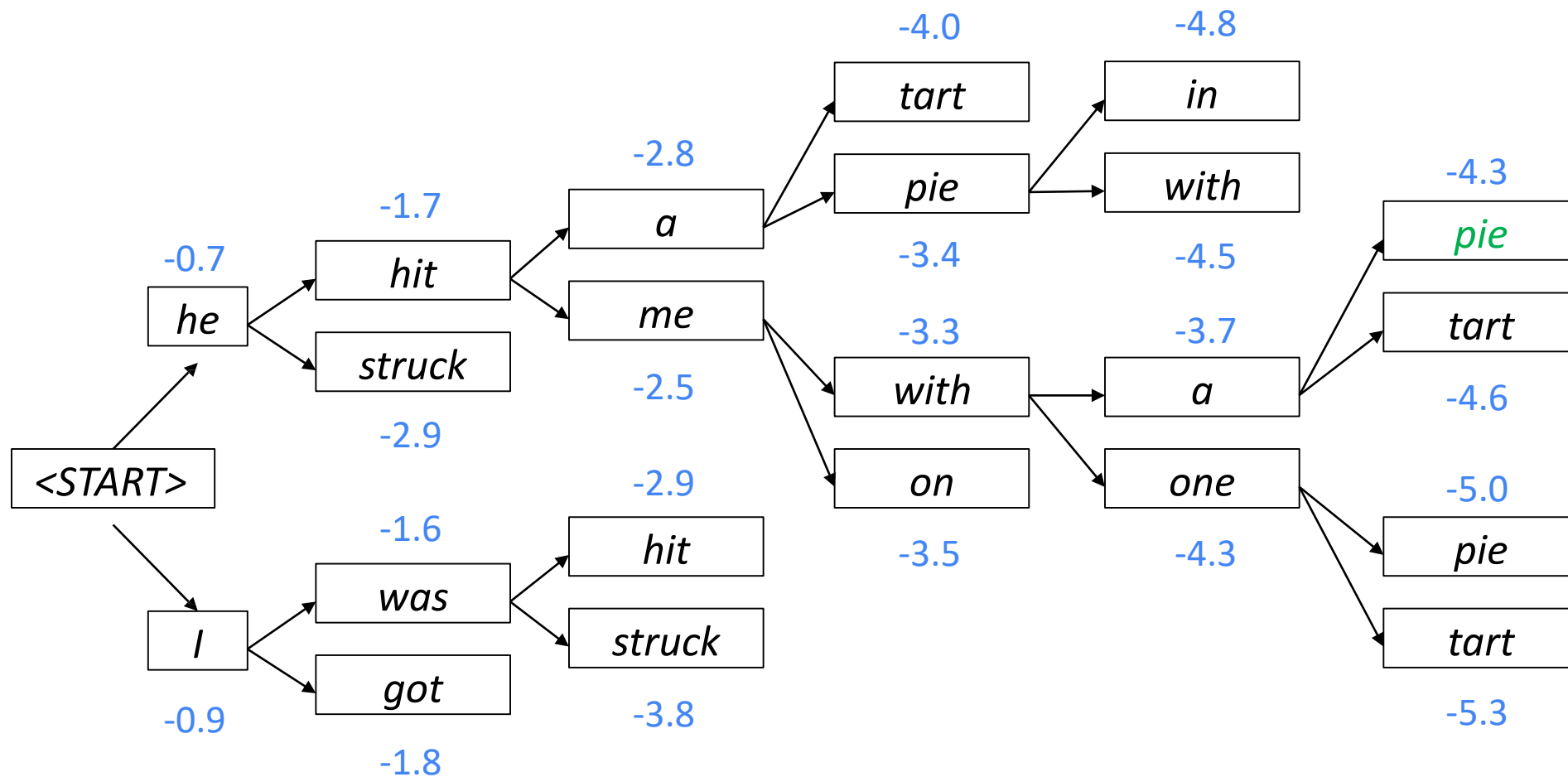


Of these k^2 hypotheses,
just keep k with highest scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$

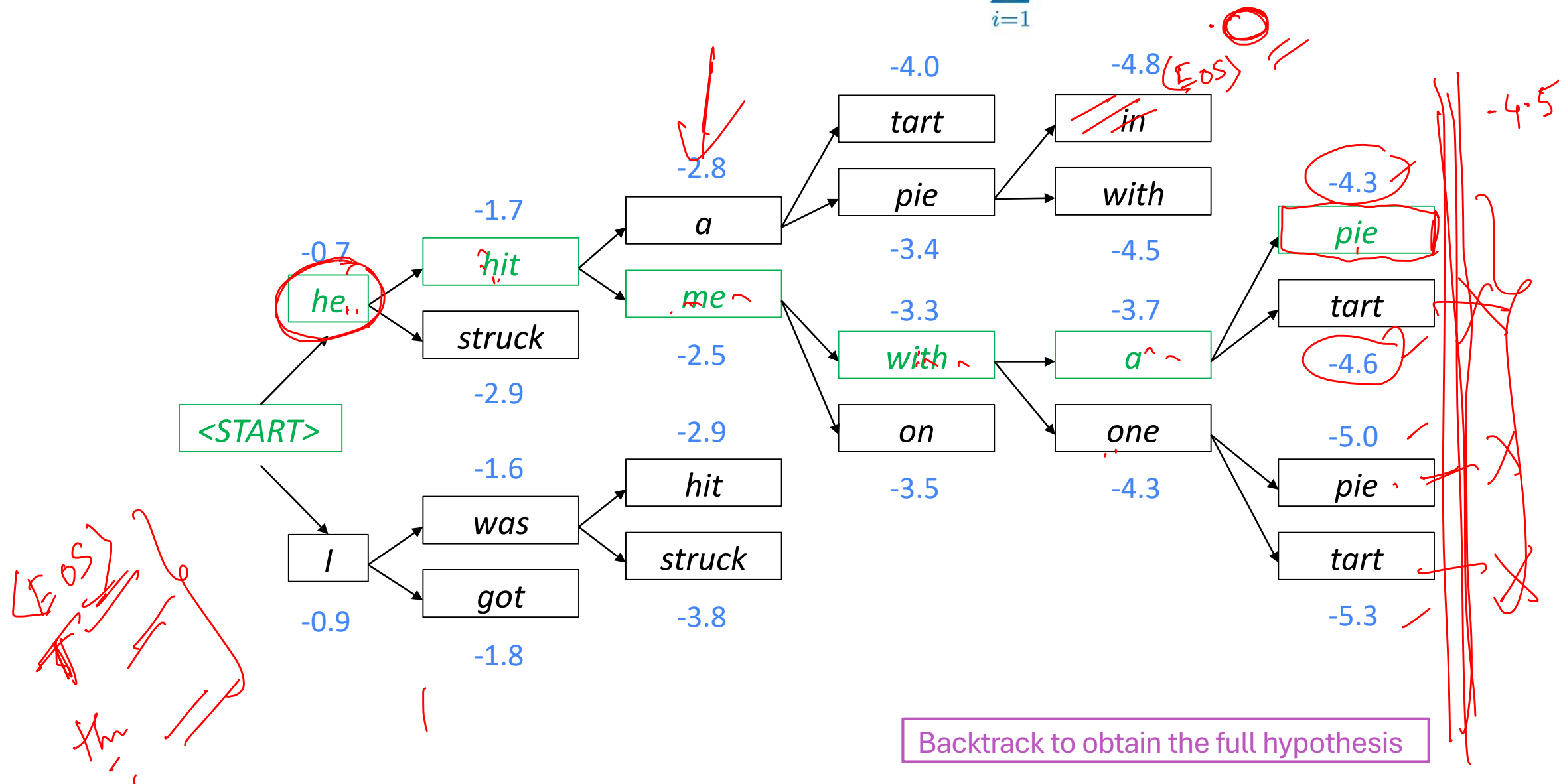


Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



This is the top-scoring hypothesis!

Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



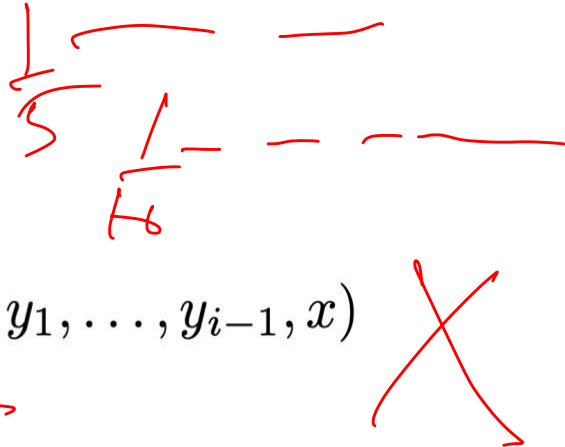
Beam Search Decoding: Stopping Criterion

- In **greedy decoding**, usually we decode until the model produces a **<END>** token
 - **For example:** <START> he hit me with a pie **<END>**
- In **beam search decoding**, different hypotheses may produce **<END>** tokens on **different timesteps**
 - When a hypothesis produces <END>, that hypothesis is **complete**.
 - **Place it aside** and continue exploring other hypotheses via beam search.
- Usually we continue beam search until:
 - We reach timestep T (where T is some pre-defined cutoff), or
 - We have at least n completed hypotheses (where n is pre-defined cutoff)



Beam Search Decoding: Finishing Up

- We have our list of completed hypotheses.
- How to select top one with highest score?
- Each hypothesis $y_1, \dots, y_t$ on our list has a score

$$\text{score}(y_1, \dots, y_t) = \log P_{\text{LM}}(y_1, \dots, y_t | x) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$$


- **Problem:** longer hypotheses have lower scores
- **Fix:** Normalize by length. Use this to select the top one instead:

$$\frac{1}{t} \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$$



Issues With RNN

- Linear interaction distance
- Bottleneck problem
- Lack of parallelizability

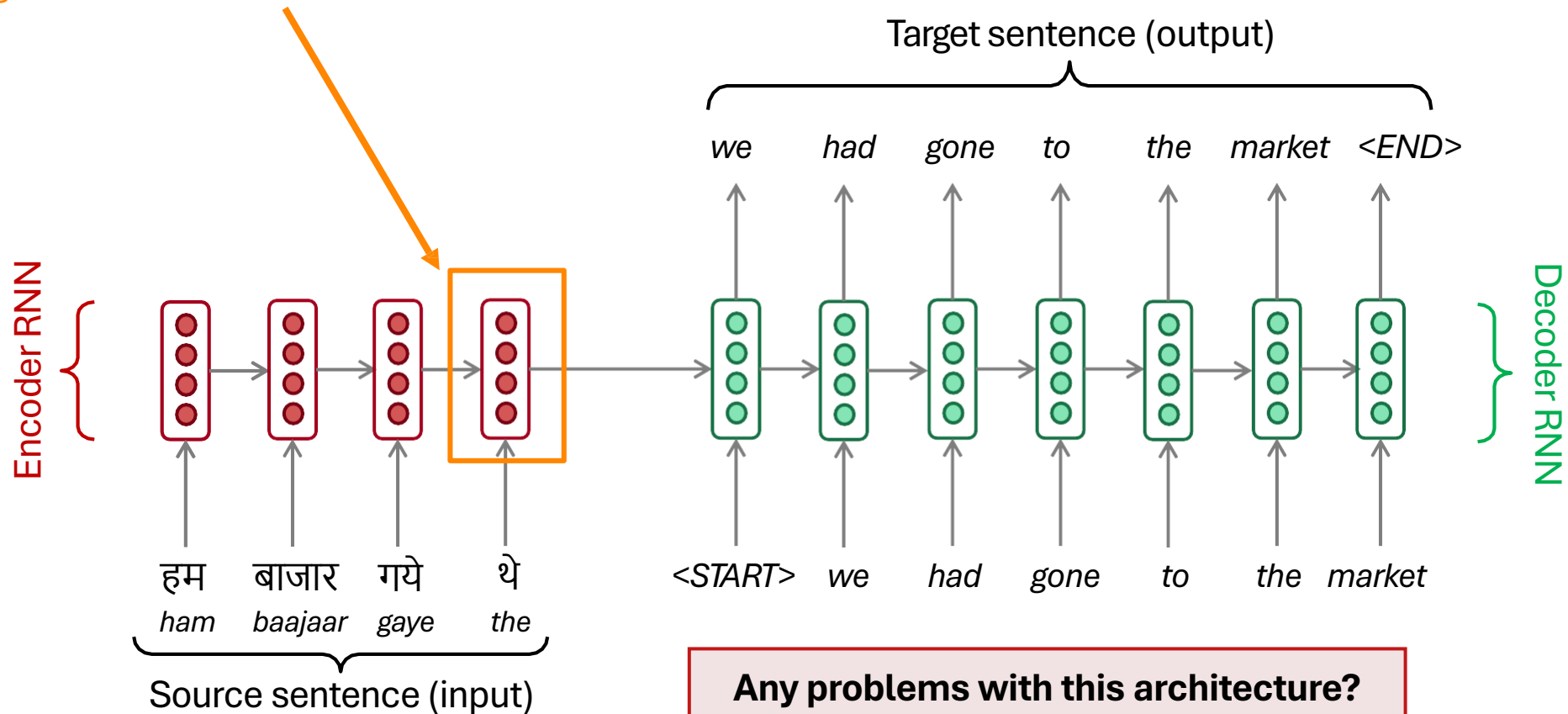
ATTENTION



Attention

Sequence-to-Sequence: The Bottleneck Problem

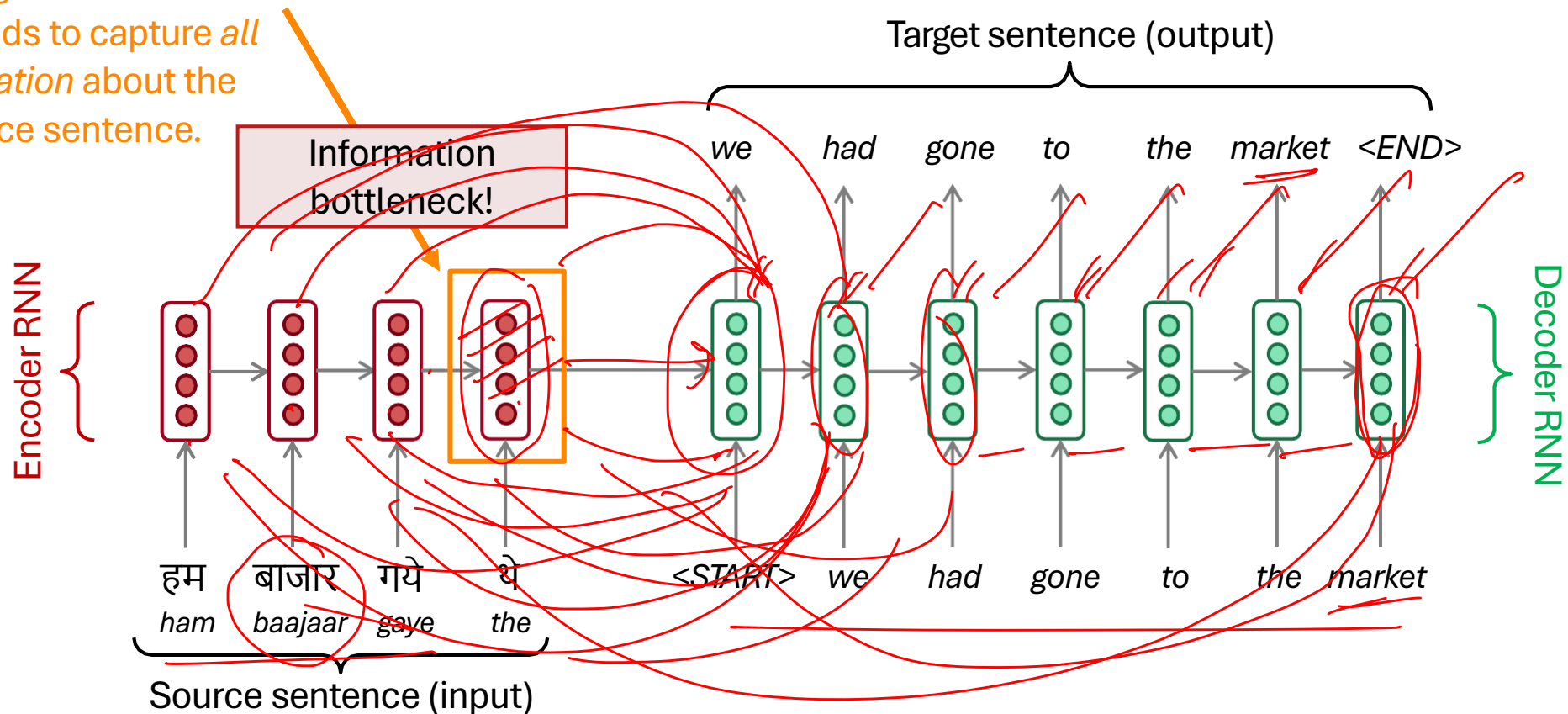
Encoding of the source sentence



Sequence-to-Sequence: The Bottleneck Problem

Encoding of the source sentence

This needs to capture *all* information about the source sentence.

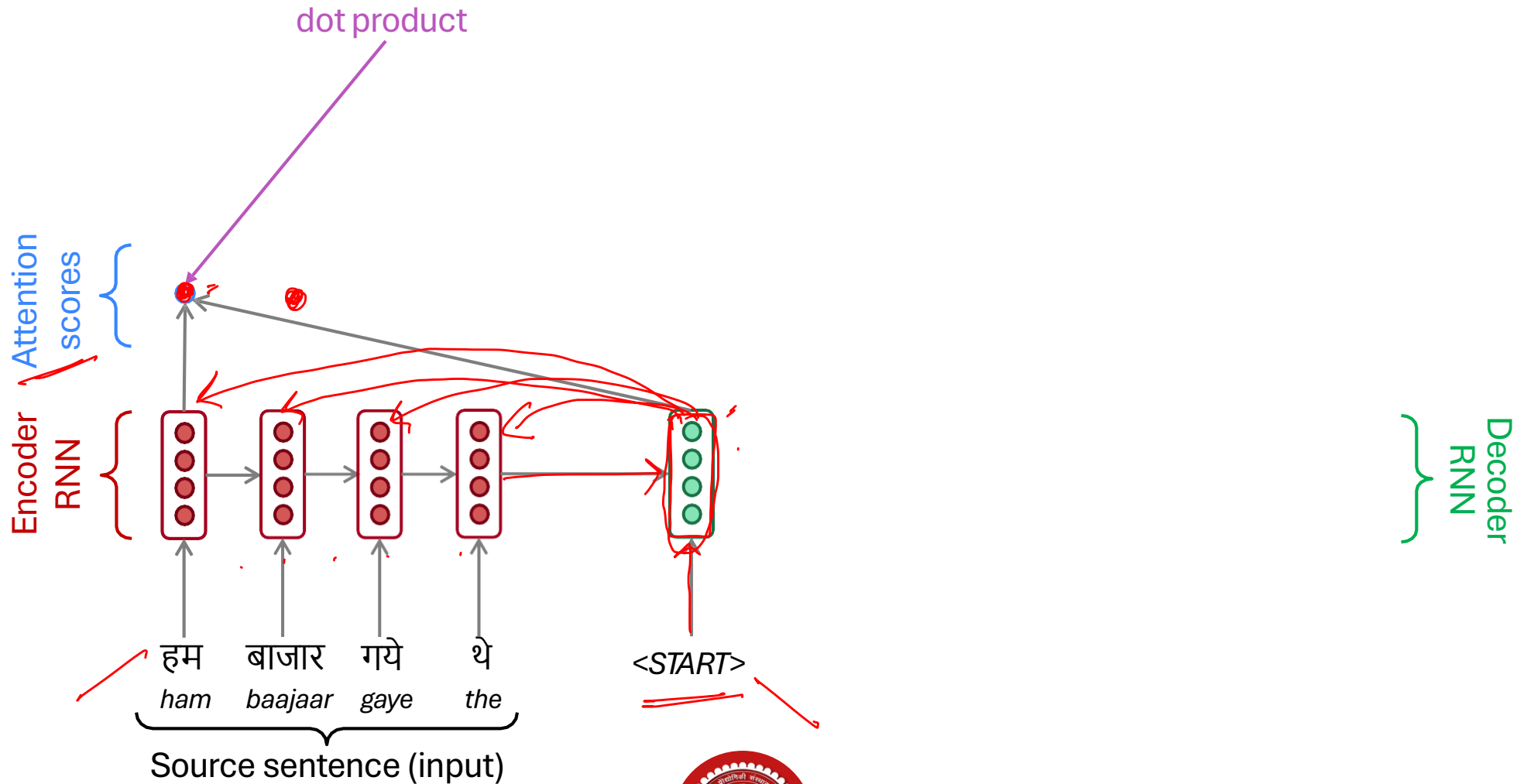


Attention

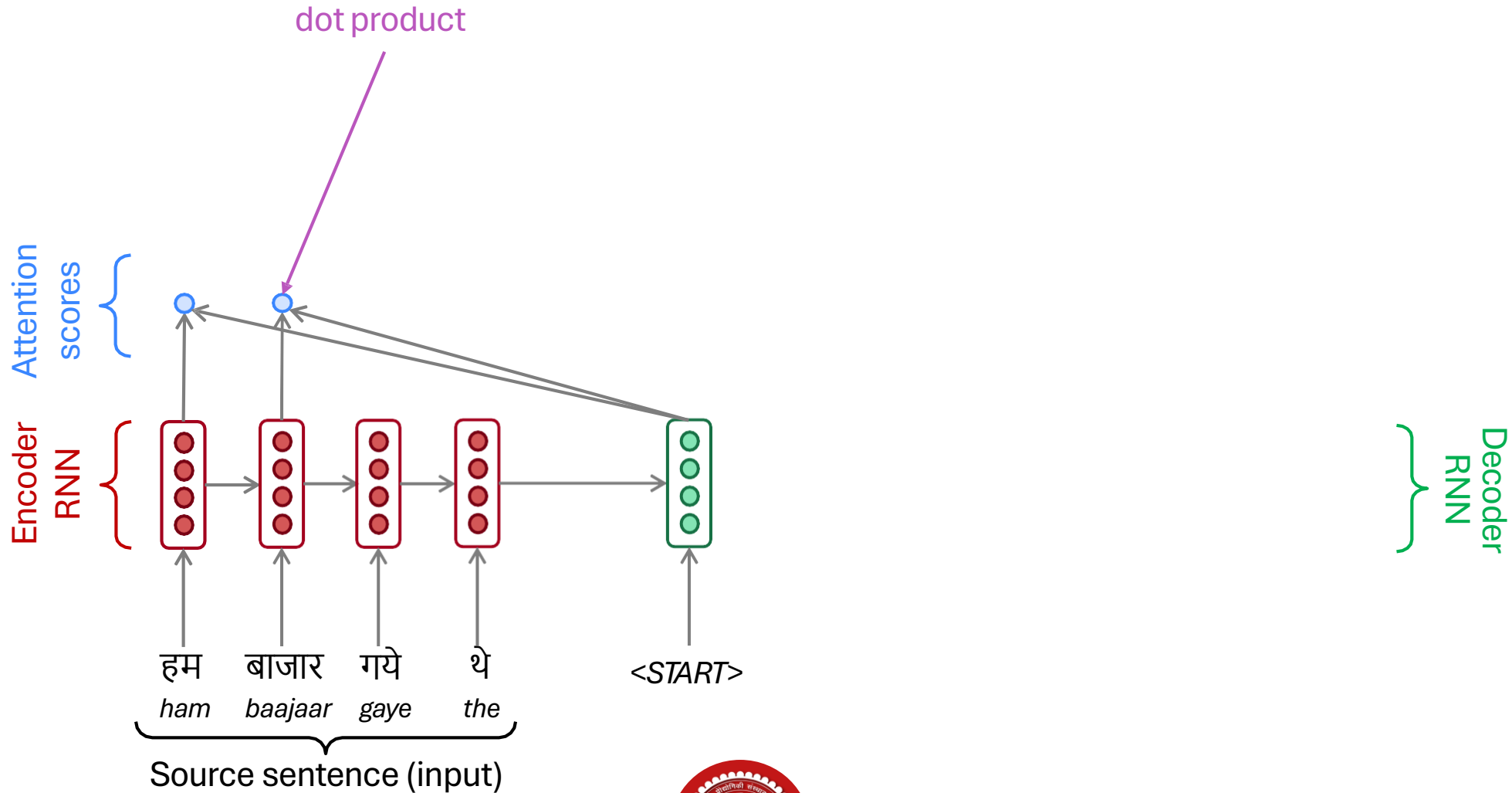
- **Attention** provides a solution to the bottleneck problem.
- **Core idea:** on each step of the decoder, use **direct connection to the encoder** to **focus on a particular part of the source sequence**
- Let's start with the visualization of the attention mechanism.



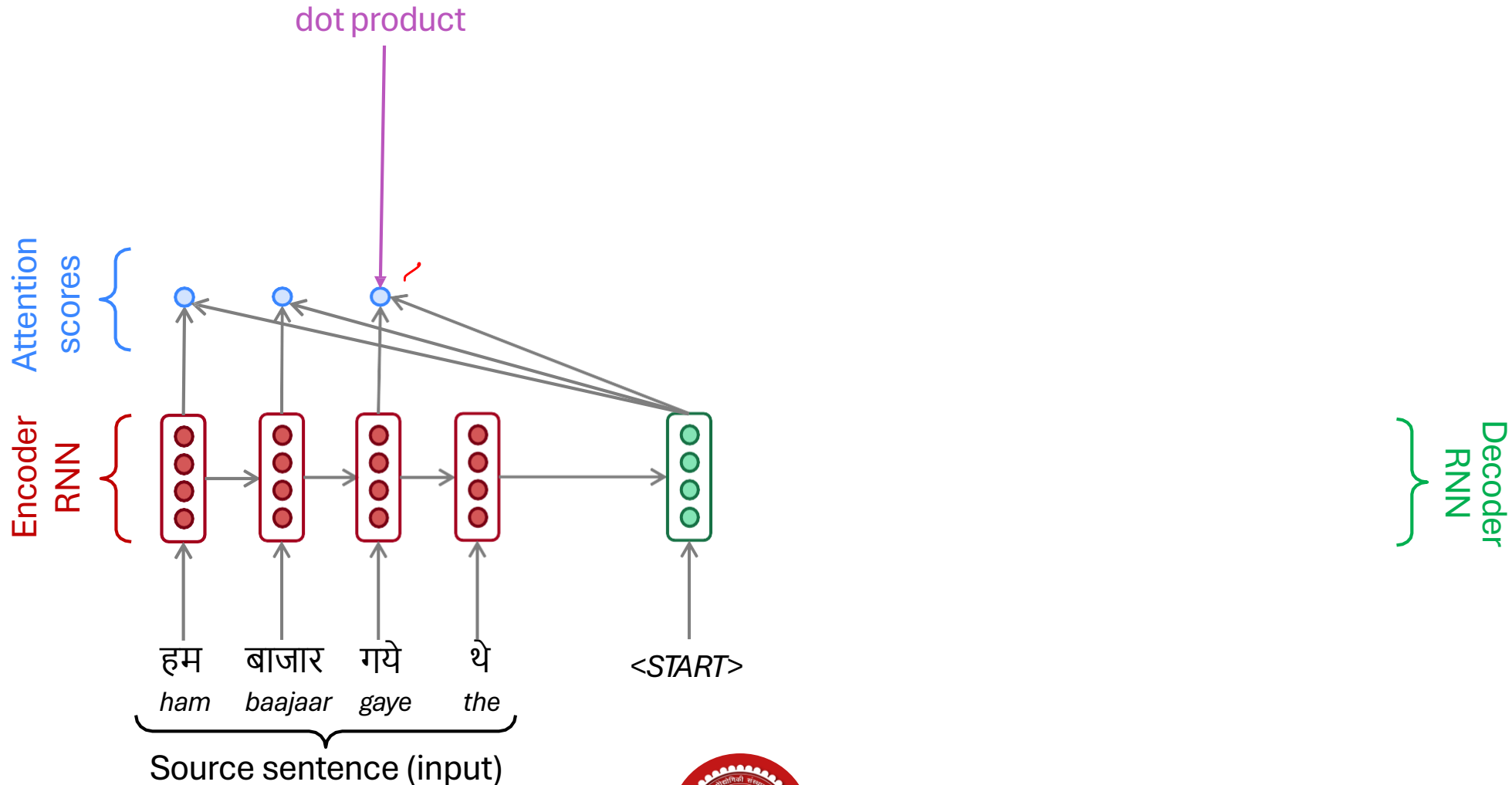
Sequence-to-Sequence With Attention



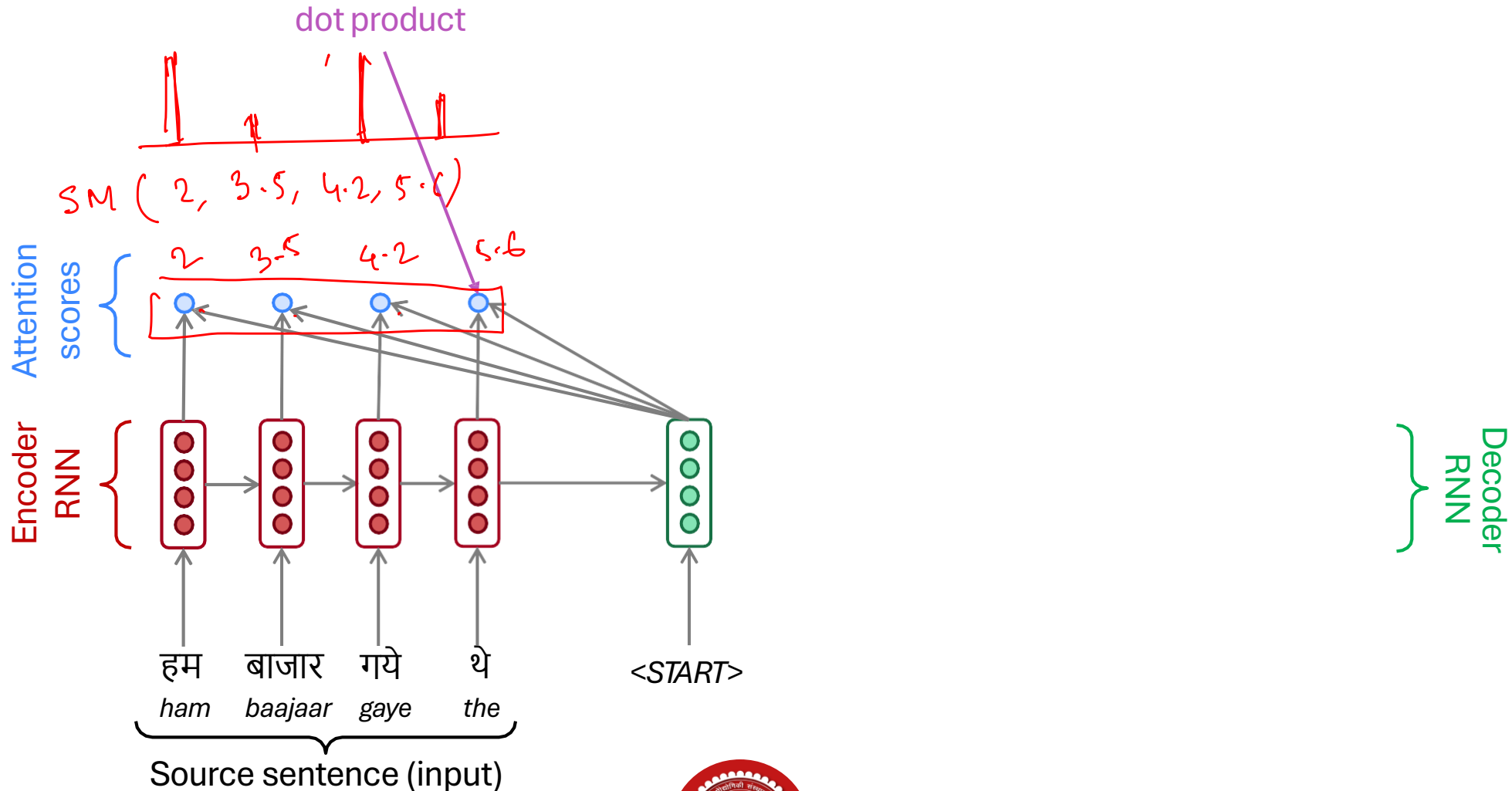
Sequence-to-Sequence With Attention



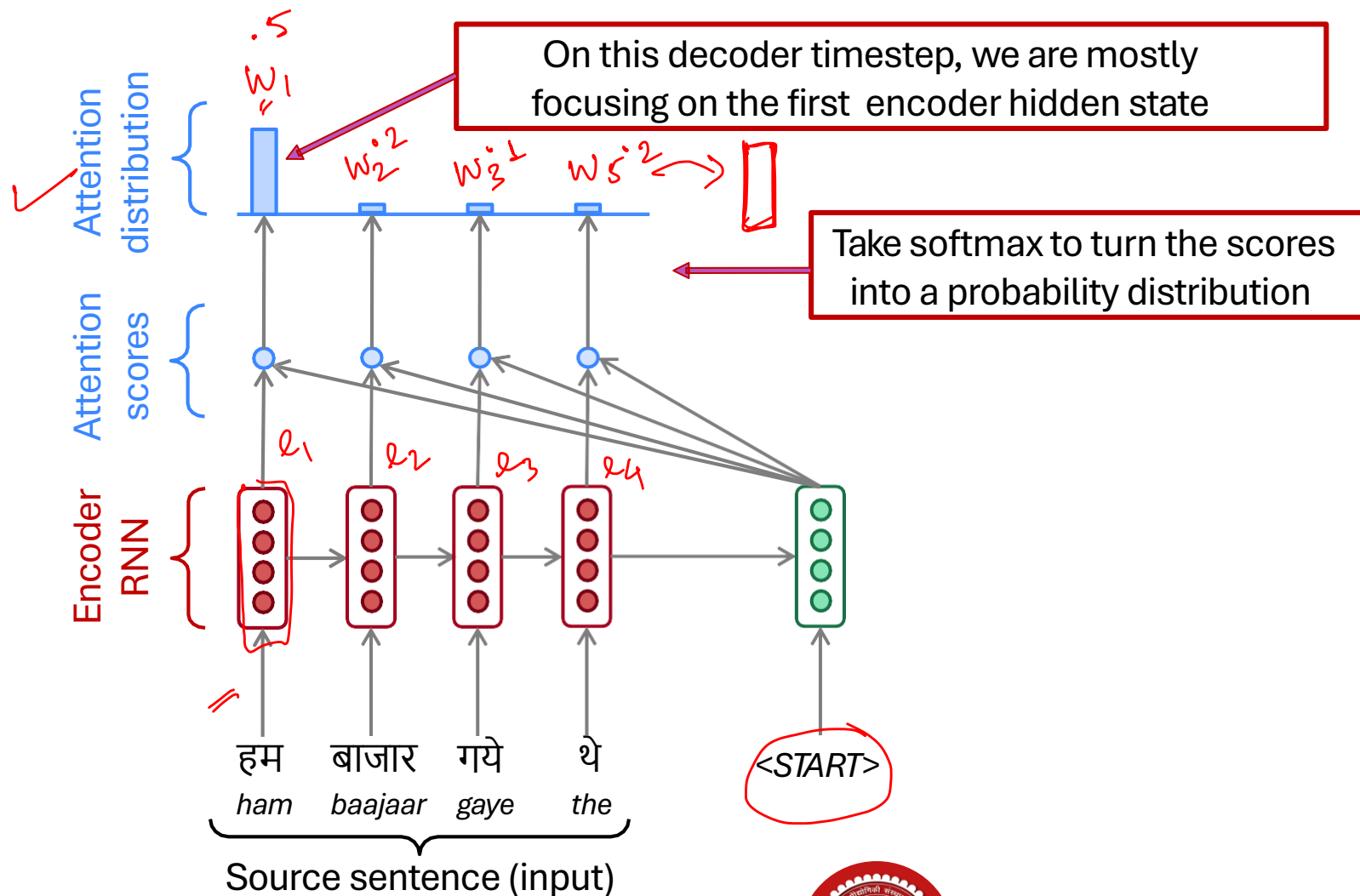
Sequence-to-Sequence With Attention



Sequence-to-Sequence With Attention



Sequence-to-Sequence With Attention



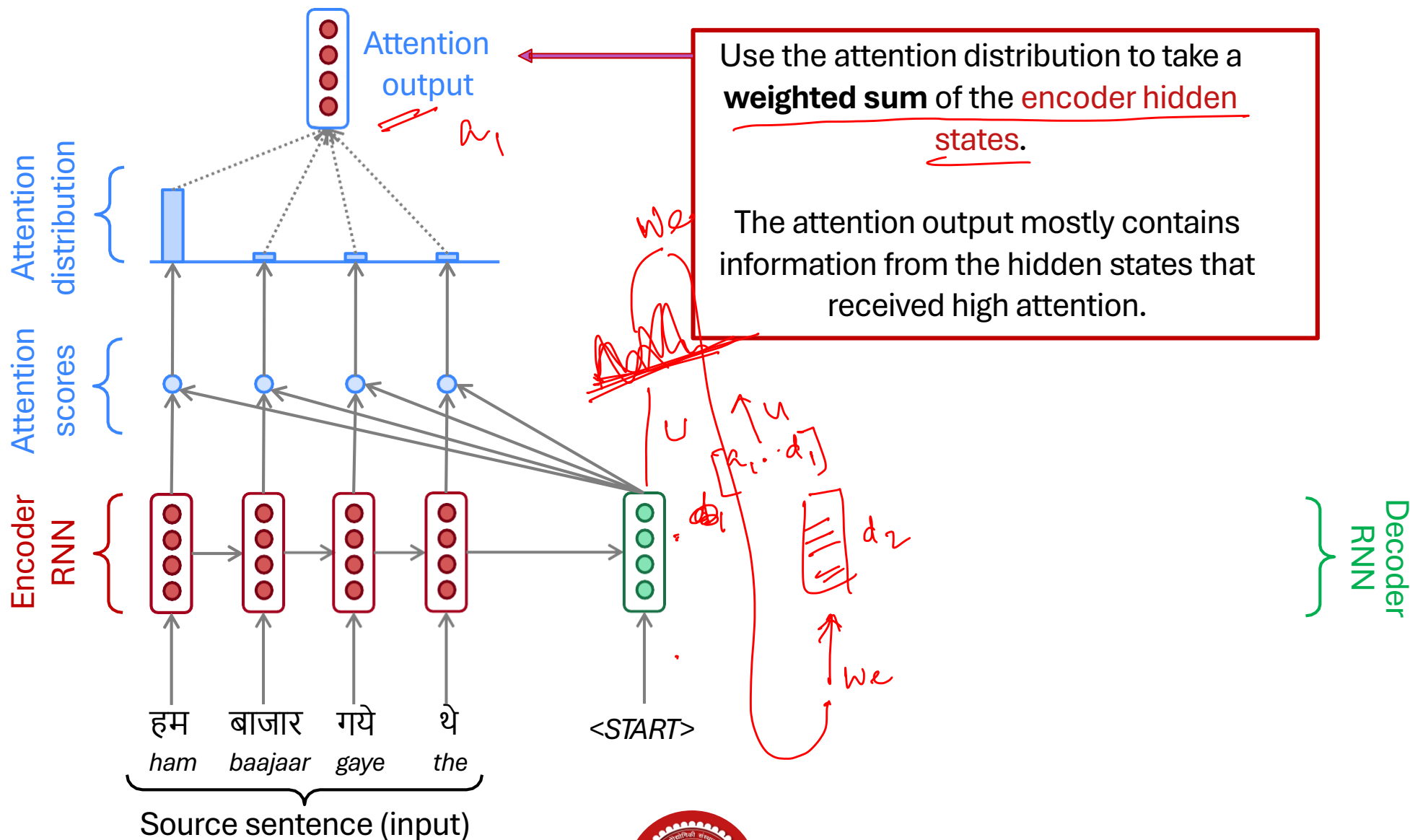
Attens output $\rightarrow$ weight sum of encoder state

$$.5 \times e_1 + .2 \times e_2 + .1 \times e_3 + .2 \times e_4$$

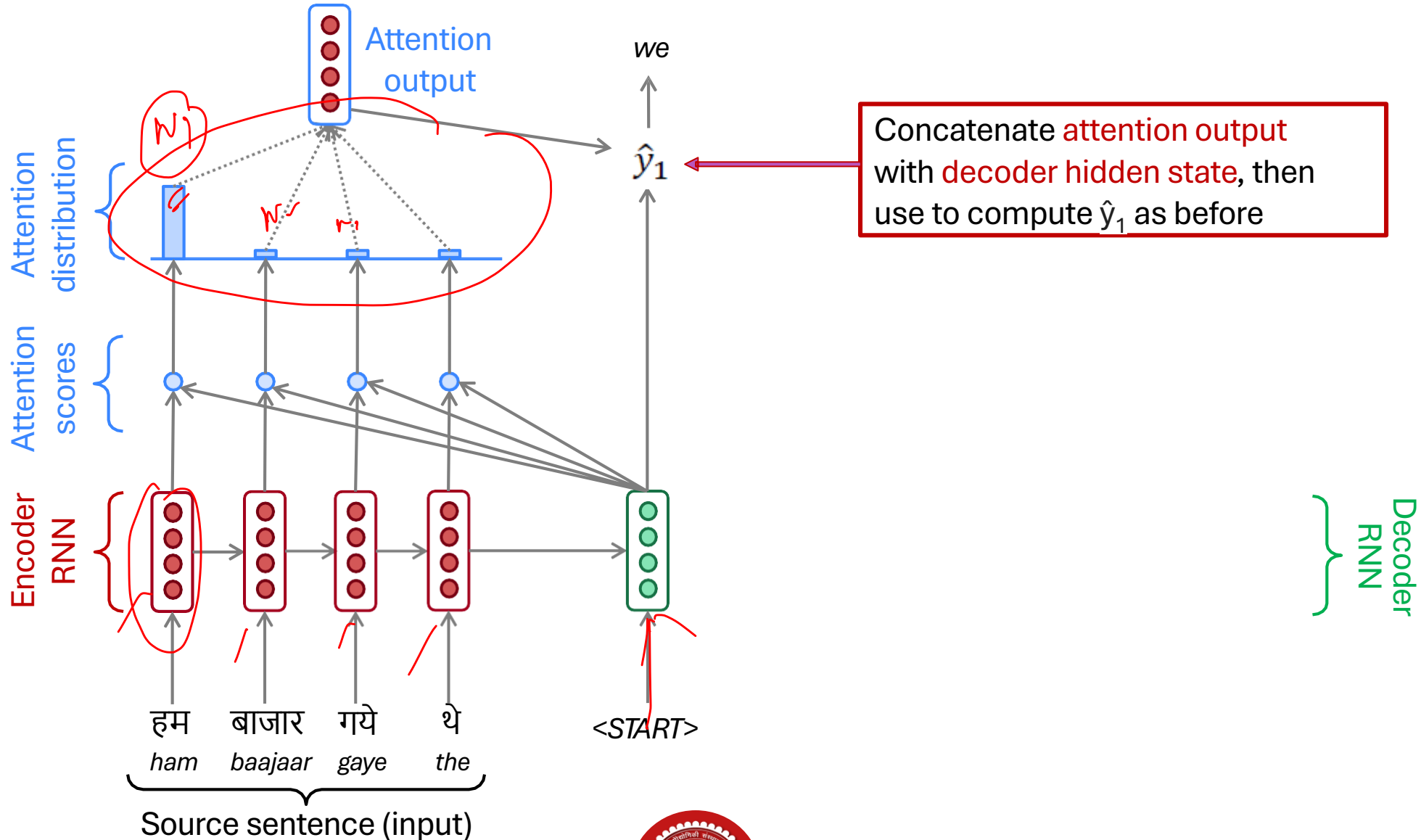
Decoder RNN



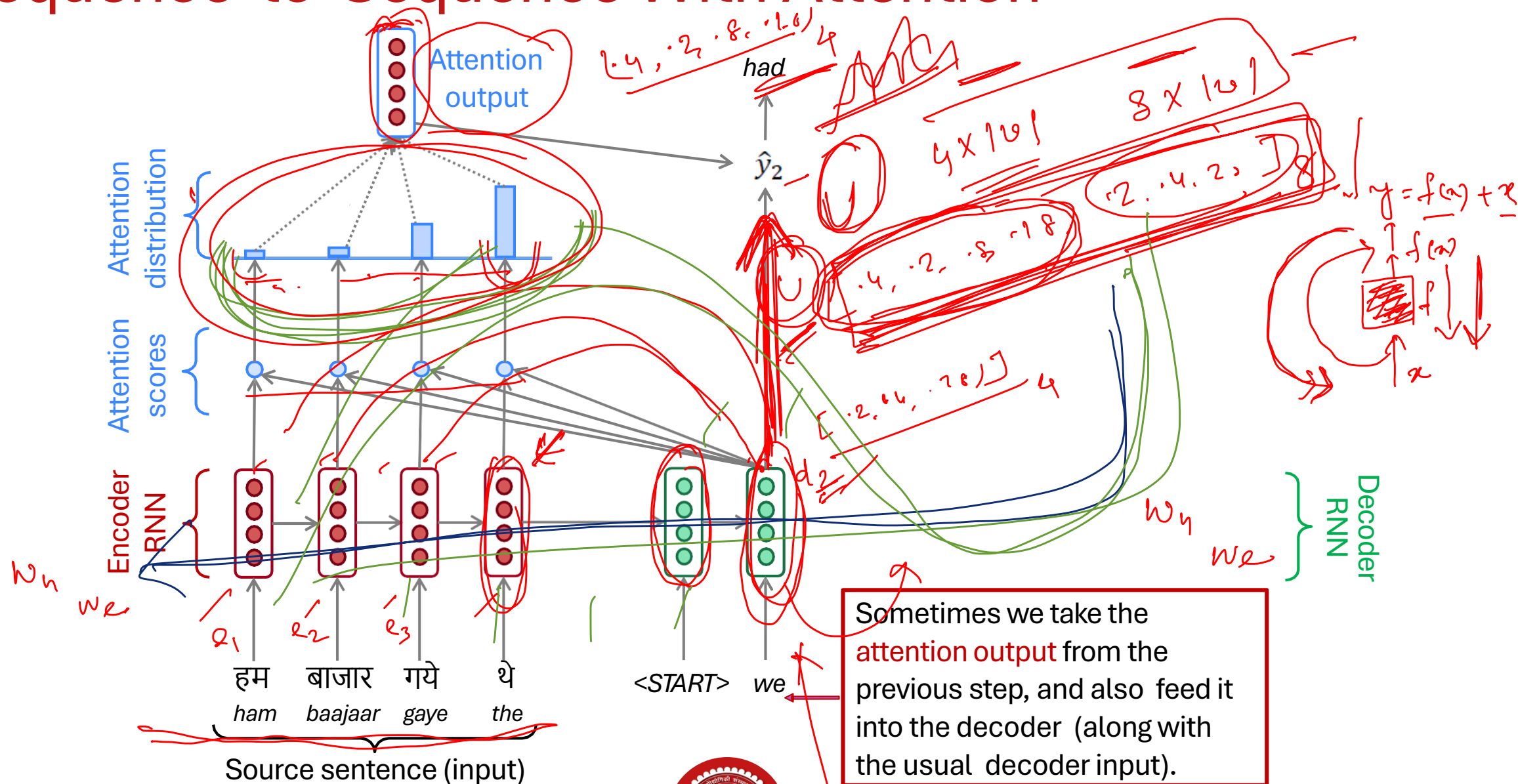
Sequence-to-Sequence With Attention

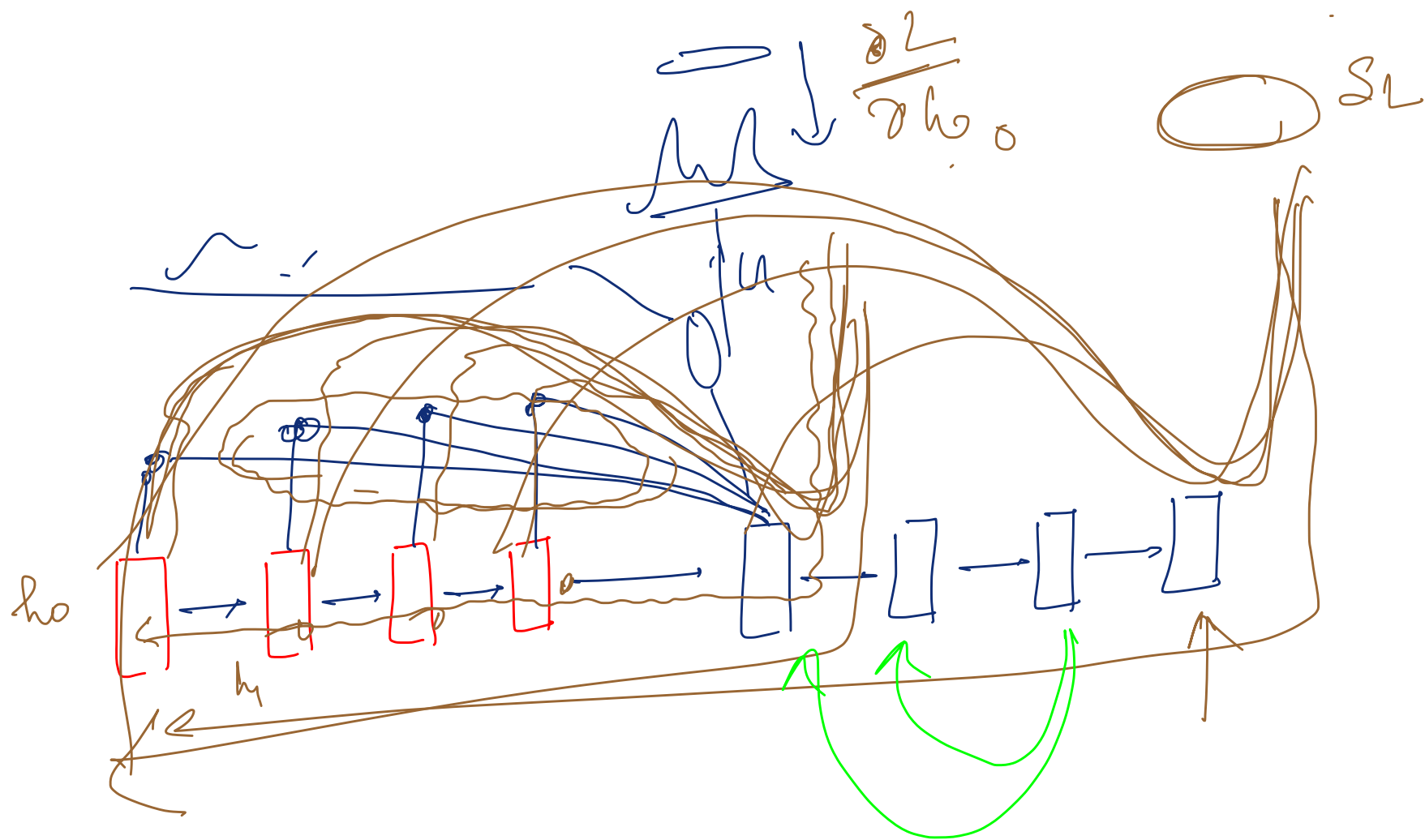


Sequence-to-Sequence With Attention

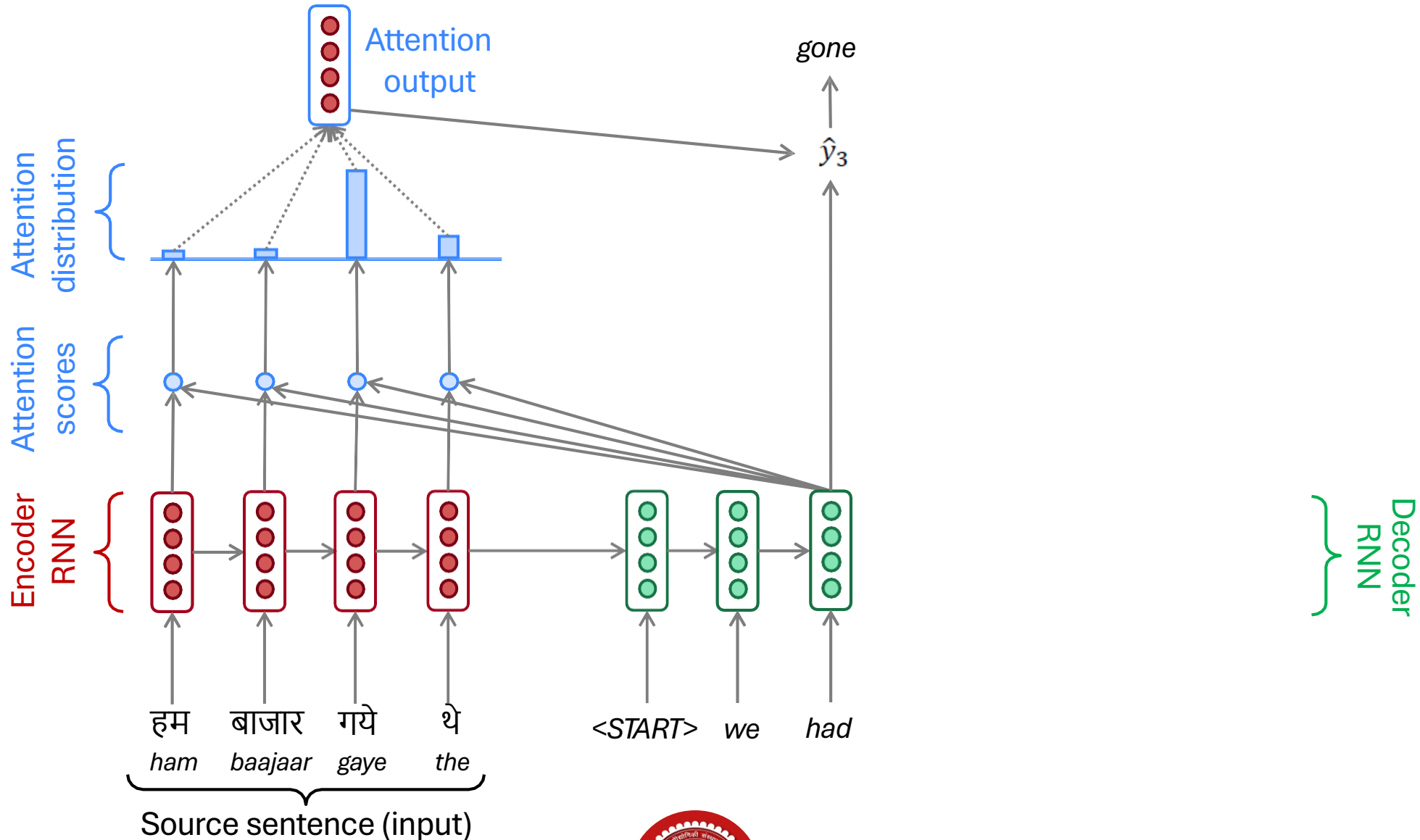


Sequence-to-Sequence With Attention

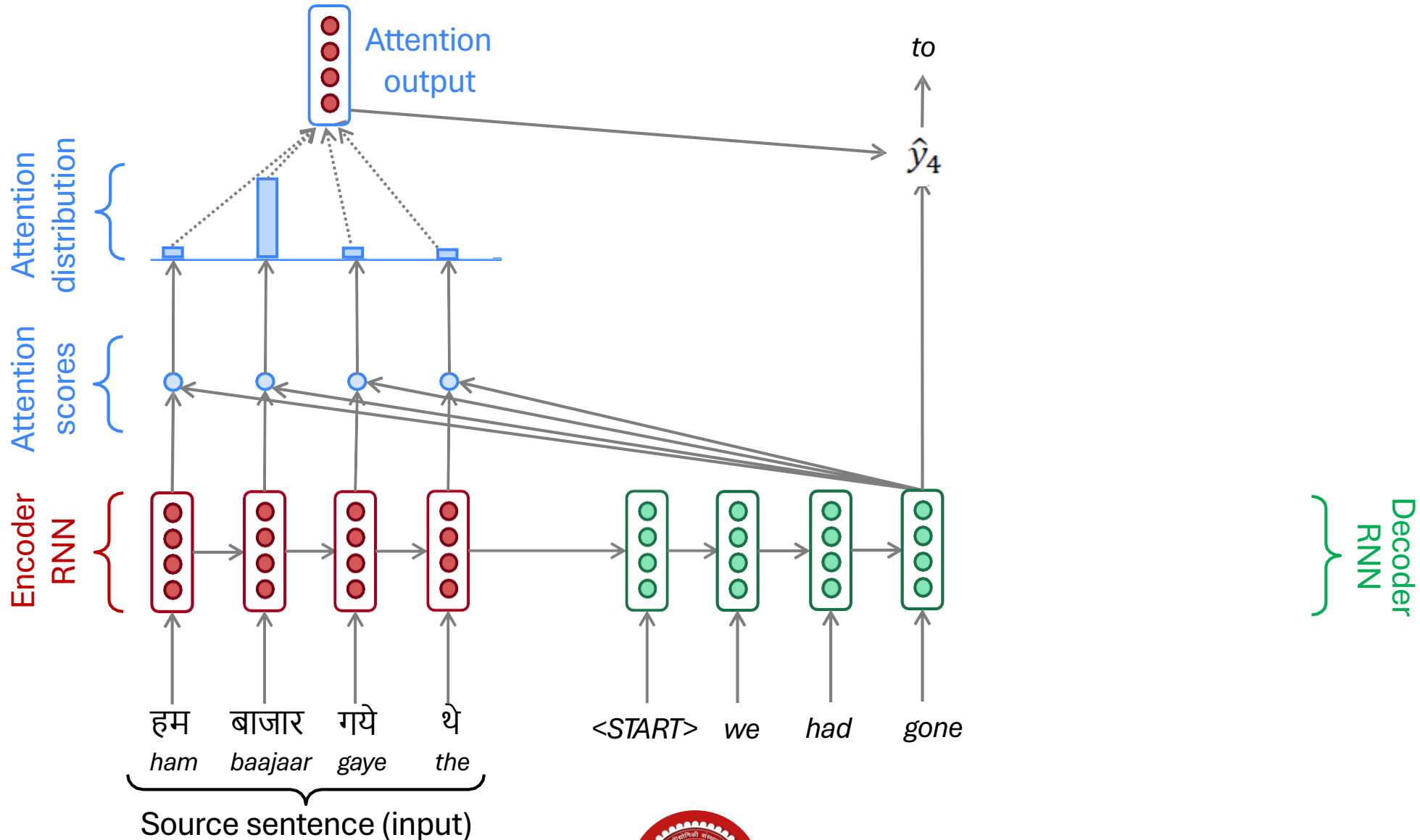




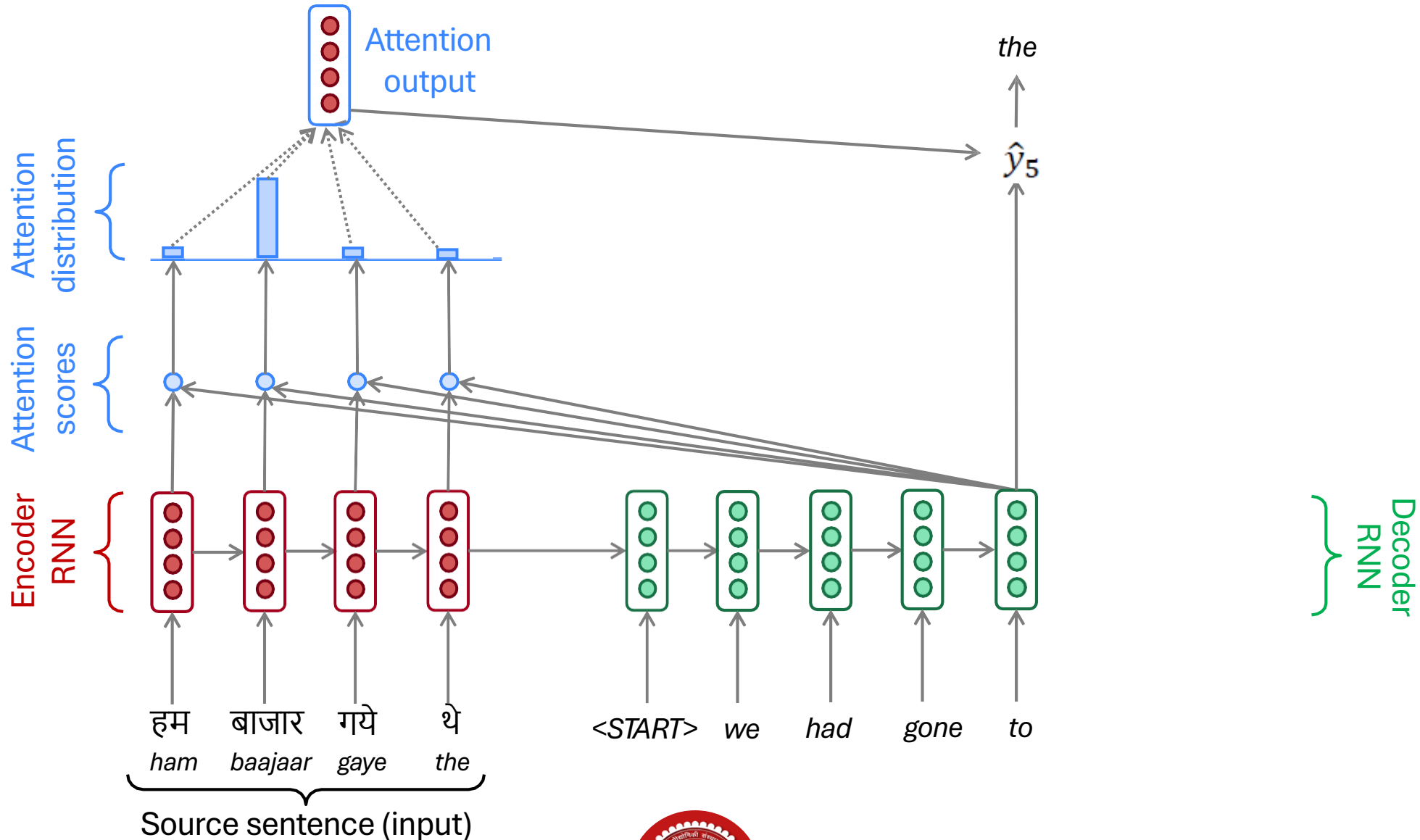
Sequence-to-Sequence With Attention



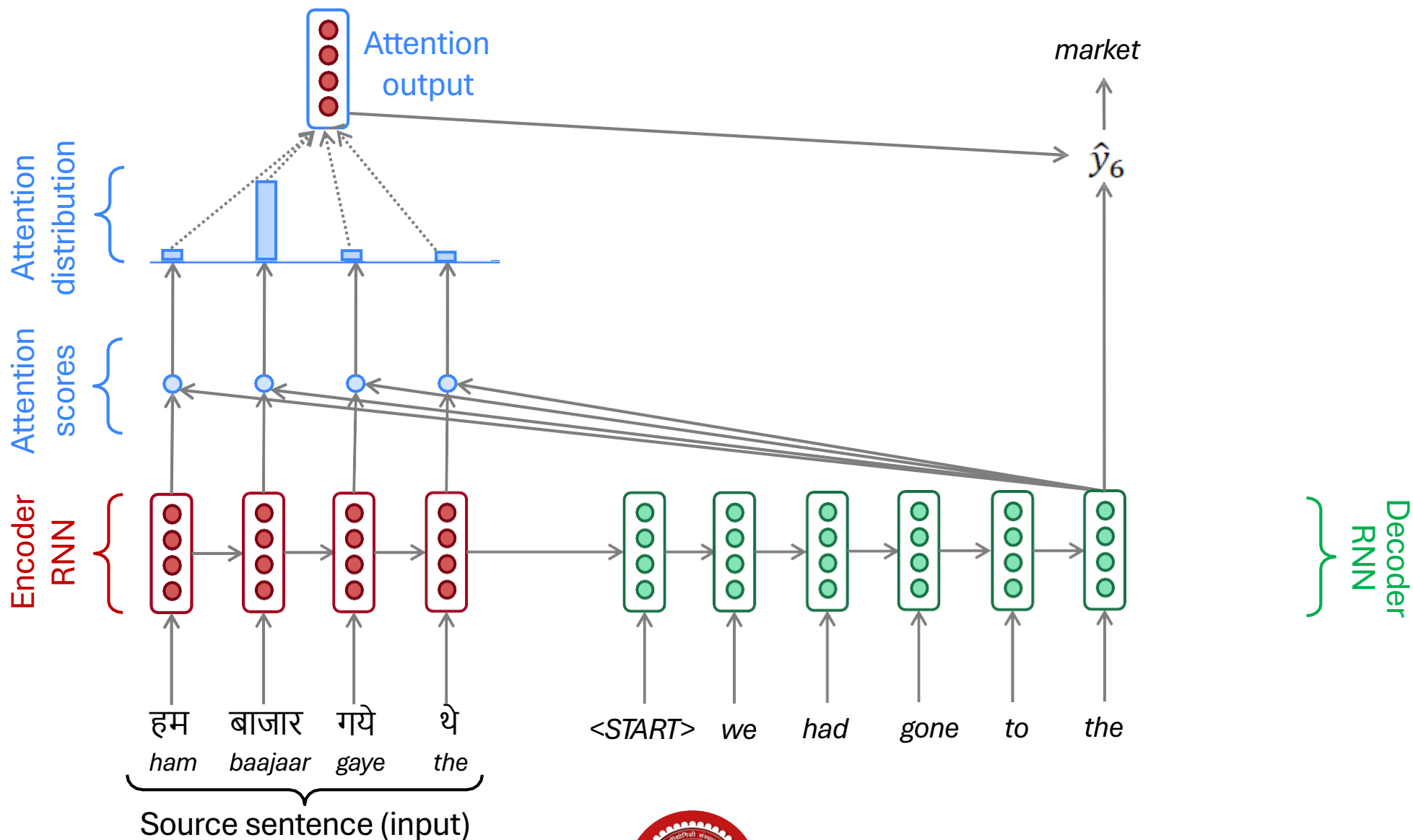
Sequence-to-Sequence With Attention

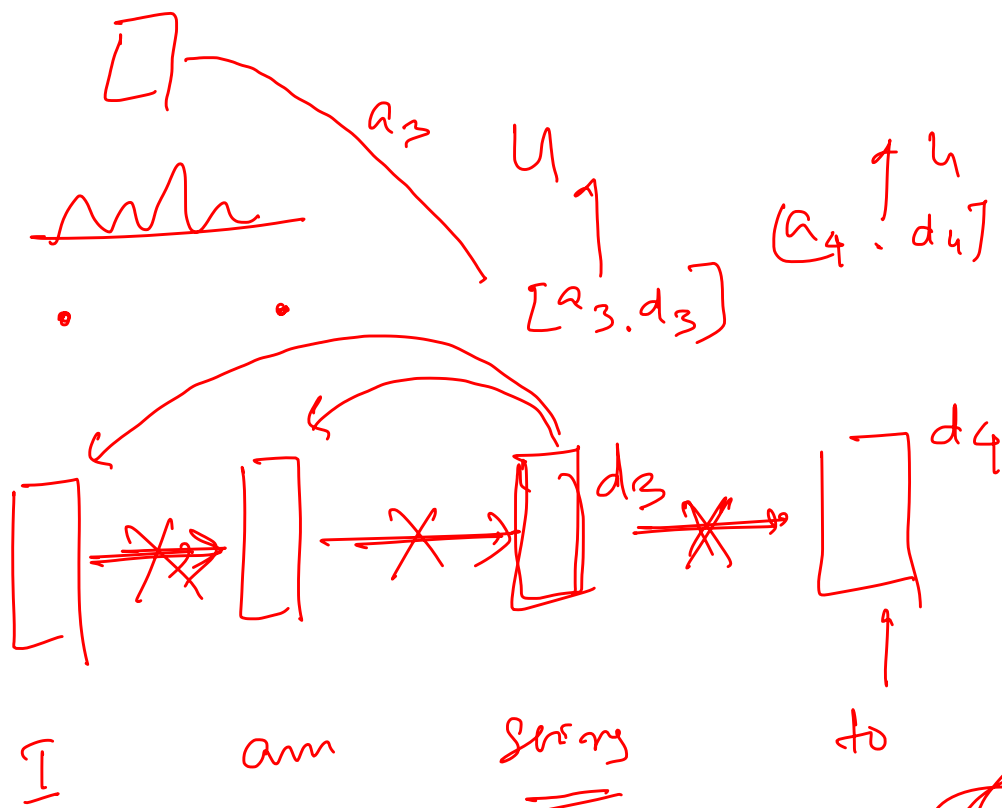


Sequence-to-Sequence With Attention

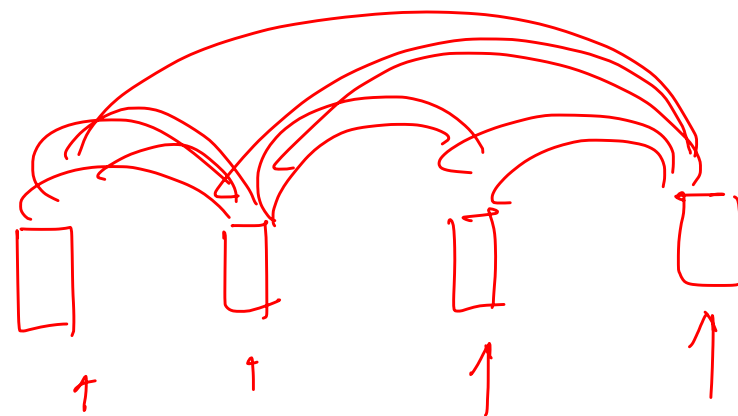


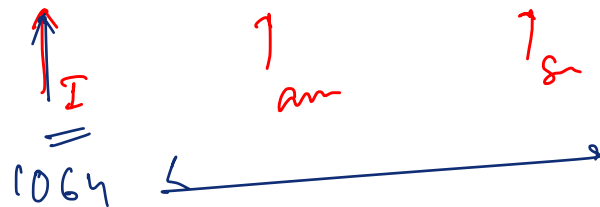
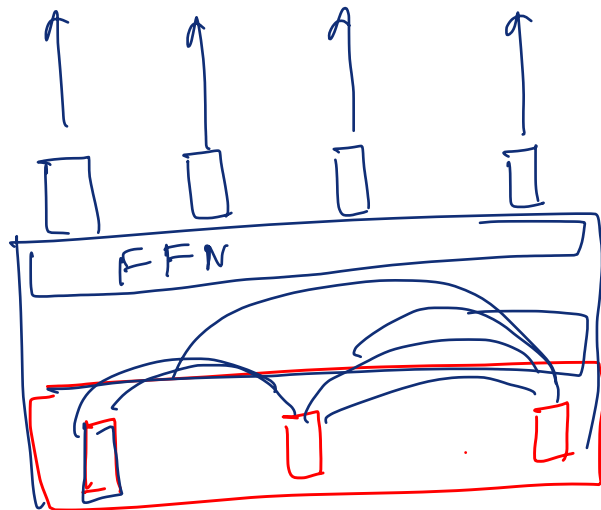
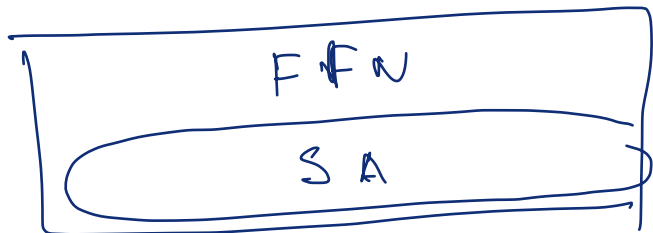
Sequence-to-Sequence With Attention





$\begin{cases} W_a = \text{Ques} \\ W_k = \text{key} \\ W_v = \text{Val} \end{cases}$





W_{FFN}^2
 W_Q^2
 W_K^2
 W_V^2



$W_{FFN}^1 =$
 $SA \quad W_Q^1 \quad \xrightarrow{1064 \times 1064 \times 1000}$
 $W_K^1 \quad \xrightarrow{1064 \times 1064 \times 1000}$
 $W_V^1 \quad \xrightarrow{1064 \times 1064 \times 1000}$

Attention: In Equations

- We have encoder hidden states $h_1, \dots, h_N \in \mathbb{R}^h$
- On timestep t , we have decoder hidden state $s_t \in \mathbb{R}^h$
- We get the attention scores e^t for this step:

$$e^t = [s_t^T h_1, \dots, s_t^T h_N] \in \mathbb{R}^N$$

- We take softmax to get the attention distribution α^t for this step (this is a probability distribution, sums to 1)

$$\alpha^t = \text{softmax}(e^t) \in \mathbb{R}^N$$

- We use α^t to take a weighted sum of the encoder hidden states to get the attention output a_t

$$a_t = \sum_{i=1}^N \alpha_i^t h_i \in \mathbb{R}^h$$

- Finally we concatenate the attention output a_t with the decoder hidden state s_t and proceed as in the non-attention seq2seq model

$$[a_t; s_t] \in \mathbb{R}^{2h}$$



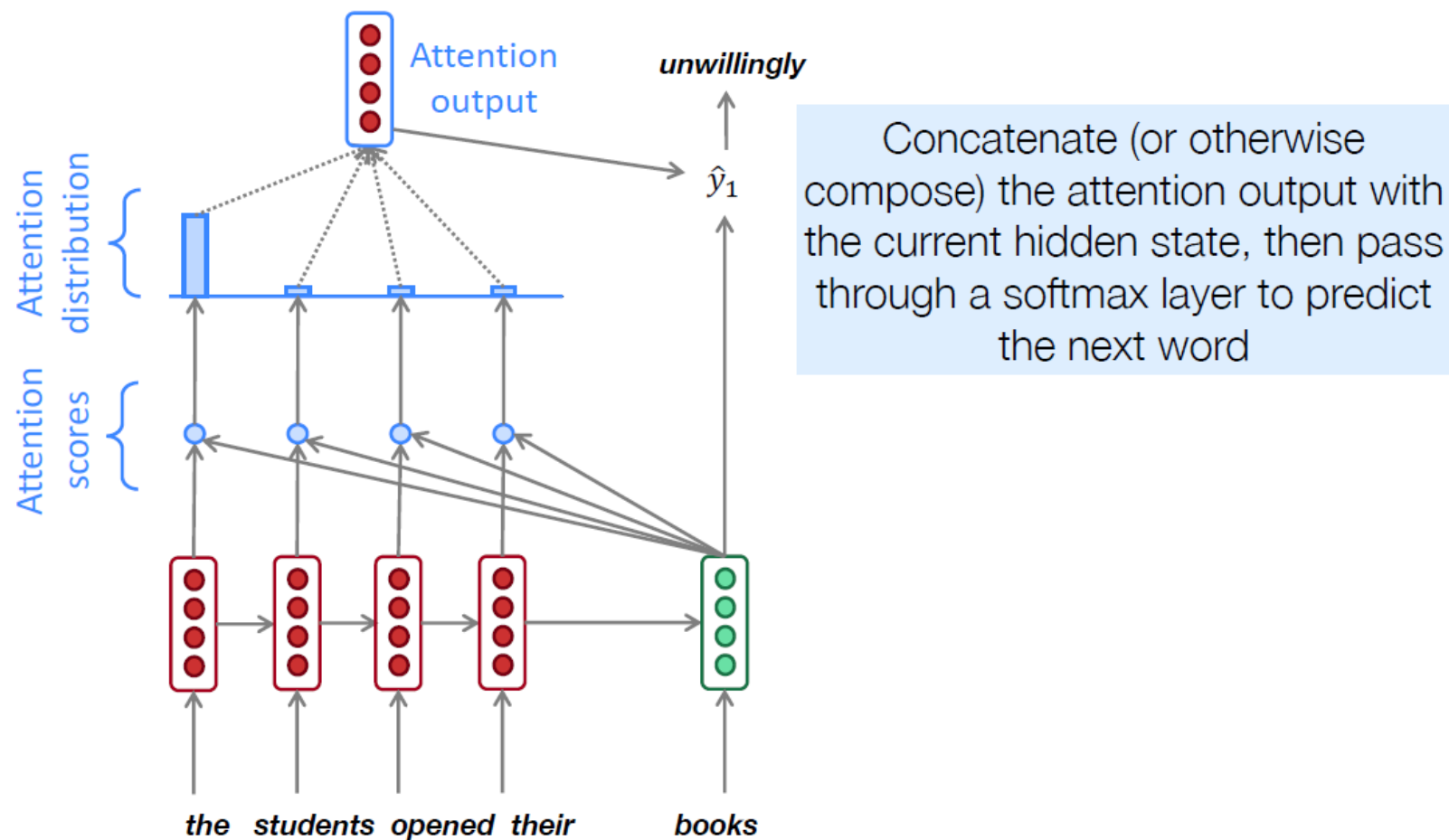
Attention is Great

- Attention significantly **improves NMT performance**
 - It's very useful to allow decoder to focus on certain parts of the source
- Attention **solves the bottleneck problem**
 - Attention allows decoder to look directly at source; bypass bottleneck
- Attention **helps with vanishing gradient problem**
 - Provides shortcut to faraway states
- Attention provides **some interpretability**
 - By inspecting attention distribution, we can see what the decoder was focusing on
 - We get (soft) **alignment for free!**
 - This is cool because we never explicitly trained an alignment system
 - The network just learned alignment by itself

	he	hit	me	with	a	pie
il						
a						
m'						
entarté						



Seq2Seq+Attention for LM



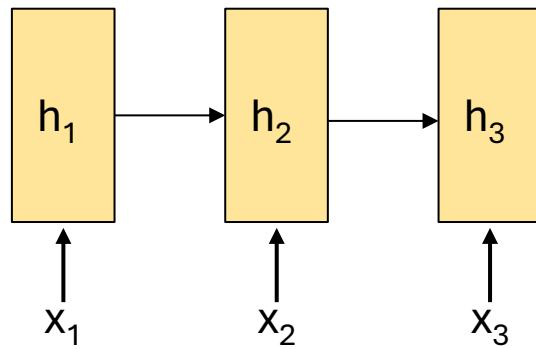
Attention is a *General* Deep Learning Technique

- We've seen that attention is a great way to improve the sequence-to-sequence model for Machine Translation.
- However: You can use attention in *many architectures* (not just seq2seq) and *many tasks* (not just MT)
- *More general definition of attention*:
 - Given a set of vector *values*, and a vector *query*, *attention* is a technique to compute a weighted sum of the values, dependent on the query.
- We sometimes say that the *query attends to the values*.
- For example, in the seq2seq + attention model, each decoder hidden state (query) *attends to* all the encoder hidden states (values).
- **Intuition**:
 - The weighted sum is a *selective summary* of the information contained in the values, where the query determines which values to focus on.
 - Attention is a way to obtain a *fixed-size representation of an arbitrary set of representations* (the values), dependent on some other representation (the query).



Attention

Encoding



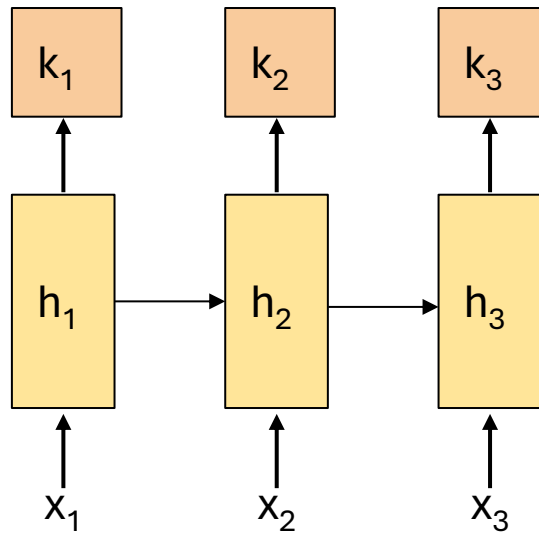
Input Sequence



Attention

Key vectors represent what **information** is **encoded** at each encoder time step.

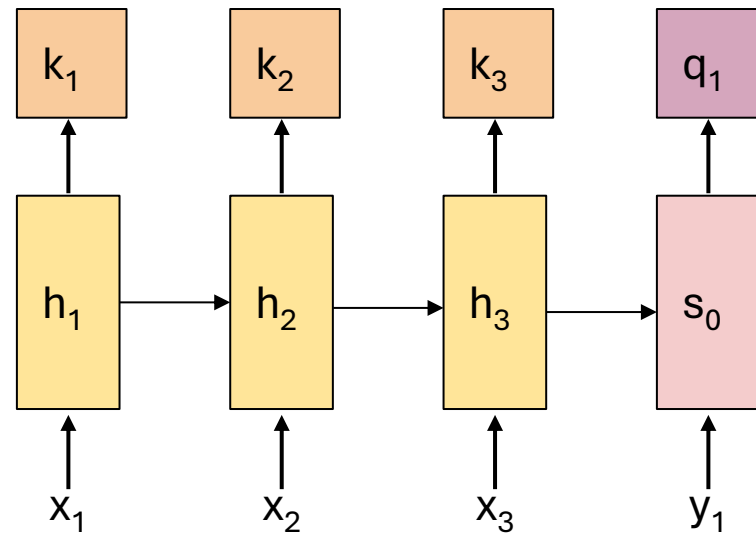
Encoding



Input Sequence



Attention



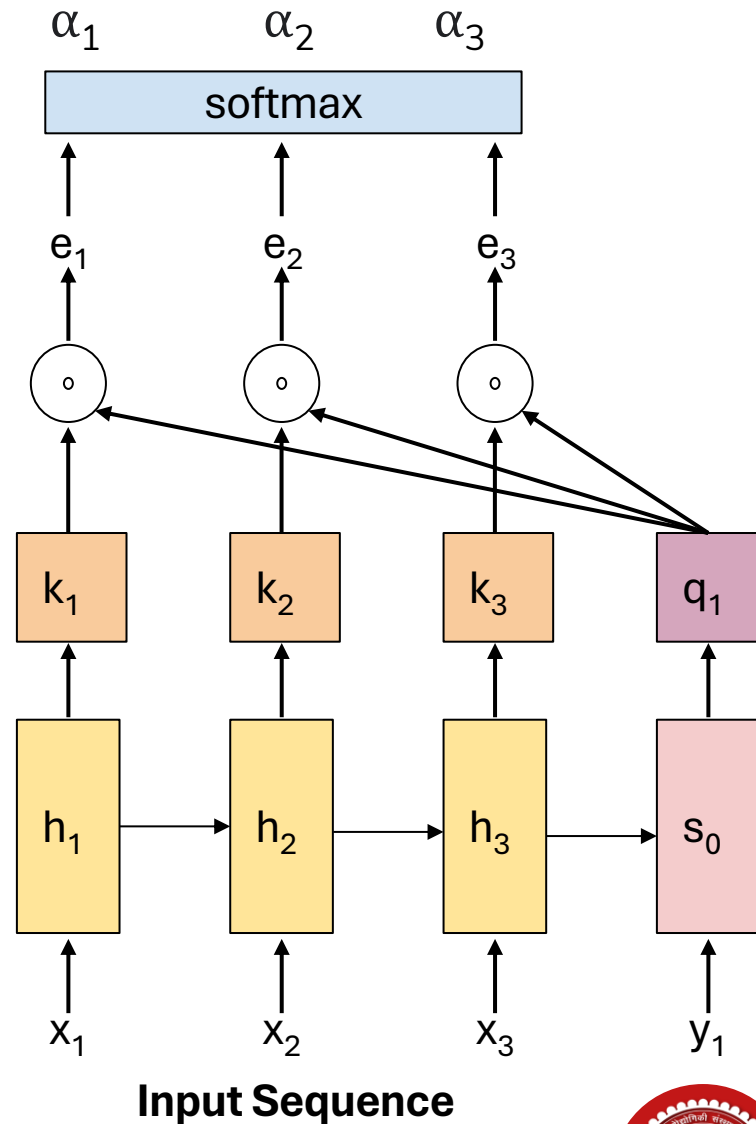
Input Sequence

Decoding

Query vectors represent what information we are **looking for** at each decoder time step.



Attention



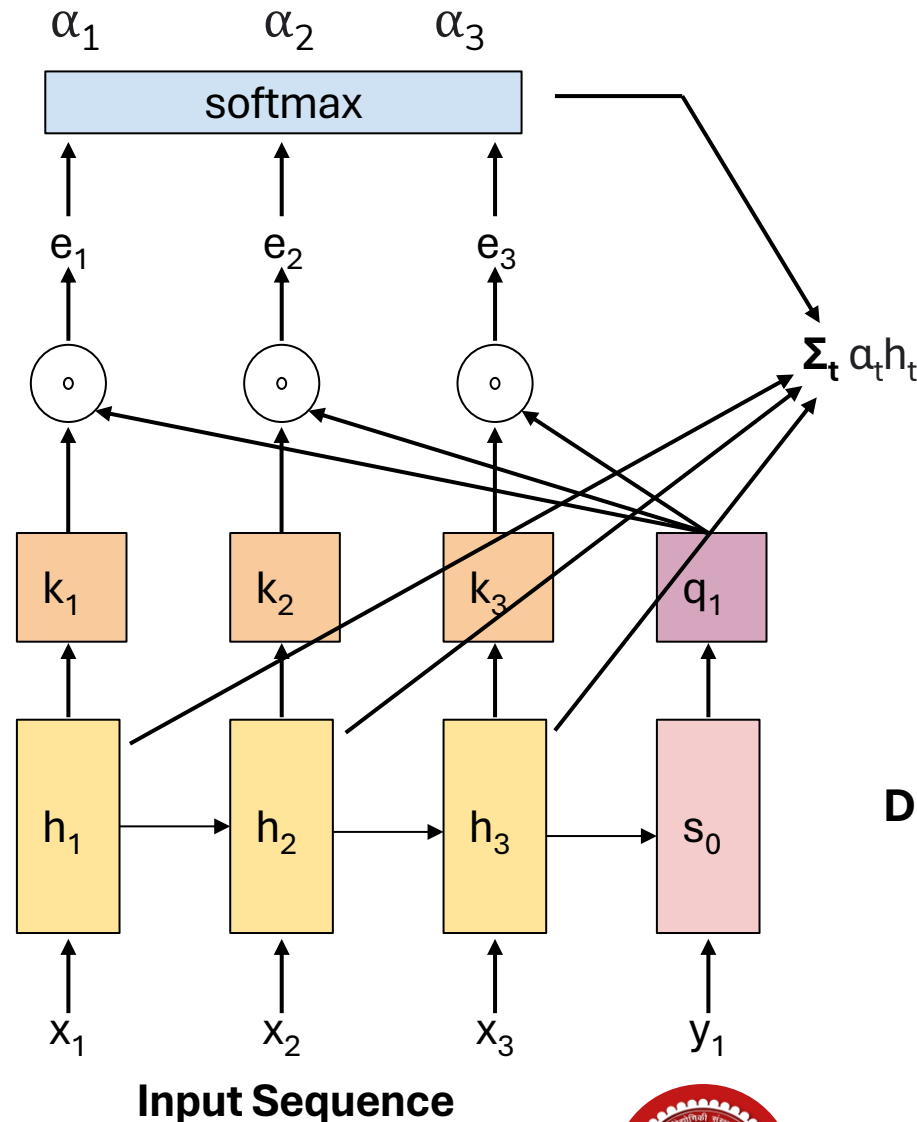
Softmax converts the similarity scores into a **probability distribution**.

Dot product between query vector and every key vector gives **similarity score**.

Decoding



Attention



The output of attention mechanism is the **weighted sum** of hidden vectors.

Instead of simply summing up the hidden vectors, we can transform them using a learned function to generate **value vectors** and then compute a weighted sum.

Decoding



Variants of Attention

- Original formulation: $a(\mathbf{q}, \mathbf{k}) = w_2^T \tanh(W_1[\mathbf{q}; \mathbf{k}])$
- Bilinear product: $a(\mathbf{q}, \mathbf{k}) = \mathbf{q}^T W \mathbf{k}$ Luong et al., 2015
- Dot product: $a(\mathbf{q}, \mathbf{k}) = \mathbf{q}^T \mathbf{k}$ Luong et al., 2015
- Scaled dot product: $a(\mathbf{q}, \mathbf{k}) = \frac{\mathbf{q}^T \mathbf{k}}{\sqrt{|\mathbf{k}|}}$ Vaswani et al., 2017

More information:

“Deep Learning for NLP Best Practices”, Ruder, 2017. <http://ruder.io/deep-learning-nlp-best-practices/index.html#attention>

“Massive Exploration of Neural Machine Translation Architectures”, Britz et al, 2017, <https://arxiv.org/pdf/1703.03906.pdf>

